BÁO CÁO ĐỒ ÁN

PROJECT 2: CODE COMMENT CLASSIFICATION

PHÂN LOẠI ĐA NHÃN CÂU CHÚ THÍCH MÃ NGUỒN TRÊN DỮ LIỆU NLBSE'23

So sánh baseline RoBERTa với các chiến lược xử lý mất cân bằng, hiệu chỉnh ngưỡng, biểu diễn theo nhãn, regularization, đồ thị tương quan nhãn và CodeBERT.

6.738 câu comment — 3 ngôn ngữ — 12 phương pháp
Cross-validation 5 fold và tập test giữ riêng

Báo cáo kết quả thực nghiệm
29/10/2026

Tóm tắt

Báo cáo nghiên cứu phân loại đa nhãn câu chú thích mã nguồn với 6.738 mẫu NLBSE'23: 2.555 câu Python, 2.418 câu Java và 1.765 câu Pharo. Mỗi ngôn ngữ được huấn luyện riêng với 5, 7 và 7 nhãn. Dữ liệu long được kiểm tra, chuyển sang wide và gom nhóm text trùng trước khi chia train/test xấp xỉ 80/20 và cross-validation 5 fold trên train.

Mười hai thí nghiệm bao gồm RoBERTa + BCE và các biến thể về loss, threshold, attention, contrastive learning, GNN, R-Drop, FGM và CodeBERT. Báo cáo sử dụng 36 run hoàn tất, cùng fingerprint dữ liệu và fold indices. Theo CV micro F1, các ứng viên đầu tiên là CodeBERT cho Python, R-Drop cho Java và Focal Loss cho Pharo. Tuy nhiên, ưu thế CV không chuyển thành ưu thế test đồng nhất: điểm test cao nhất của Python thuộc E9 v2, còn Java thuộc CodeBERT. Đây là mô tả hậu nghiệm, không phải quy tắc chọn lại model bằng test.

Phân tích cho thấy tăng recall có thể làm tăng false positive; micro và macro F1 phản ánh các mục tiêu khác nhau; kiến trúc phức tạp hơn không bảo đảm lợi ích tương xứng chi phí. Kết luận được giới hạn ở một seed, bộ chia hiện tại và ngân sách tối đa 10 epoch; chưa khẳng định vượt trội thống kê hoặc vượt baseline chính thức dưới protocol khác.

Mục lục

Tóm tắt 2

1 Bài toán và mục tiêu nghiên cứu 5

1.1 Phạm vi Project 2 5
1.2 Định nghĩa multi-label và ý nghĩa ứng dụng 5
1.3 Chiến lược và câu hỏi nghiên cứu 5

2 Phân tích khám phá dữ liệu (EDA) 6

2.1 Đơn vị mẫu, cardinality và độ thưa nhãn 6
2.2 Độ dài văn bản và token thực tế 7
2.3 Giới hạn 128 token và đặc điểm text 8
2.4 Từ vựng và giới hạn ngữ cảnh 9
2.5 Quan hệ nhãn: Python 10
2.6 Quan hệ nhãn: Java 11
2.7 Quan hệ nhãn: Pharo 12
2.8 Phân bố train/test 13
2.9 Cân bằng positive của năm fold 14
2.10 Text trùng, tập trung class và leakage 15
2.11 EDA và chiến lược thử nghiệm 16

3 Dữ liệu và quy trình tiền xử lý 17

3.1 Nguồn, cấu trúc và mất cân bằng 17
3.2 Kiểm tra raw, pivot và chuẩn hóa 18
3.3 Nhóm text trùng và giới hạn ngữ cảnh 18
3.4 Bộ chia chung và 5-fold cross-validation 20

4 Mô hình và ý nghĩa các thử nghiệm 20

4.1 Backbone và baseline chung 20
4.2 Loss: E1, E2, E3 và E7 21
4.3 Ngưỡng quyết định: E4 22
4.4 Attention, contrastive và GNN: E5, E6, E9, E12 22
4.5 Regularization và backbone: E8, E10, E11 23

5 Thiết kế đánh giá và lựa chọn model 23

5.1 Pipeline và cấu hình 23
5.2 Metric phù hợp 24
5.3 Quy tắc lựa chọn và kiểm tra triển khai 25

6 Kết quả so sánh với baseline RoBERTa 25

6.1 Python: lợi ích CV không đồng nghĩa lợi ích test 25
6.2 Java: CodeBERT nổi bật trên test, R-Drop dẫn nhẹ ở CV 27
6.3 Pharo: Focal Loss cân bằng tốt hơn, ASL ưu tiên recall 27

7 Phân tích cơ chế và insight 29

7.1 Mất cân bằng: tăng recall chưa chắc tăng hiệu quả tổng thể 29
7.2 Threshold tuning: macro tăng bằng một đánh đổi đo được 30
7.3 Attention và graph: thêm cấu trúc không bảo đảm thắng 30
7.4 Backbone và mức độ phù hợp miền 31
7.5 Hội tụ và lựa chọn epoch 31

8 Chi phí và chiến lược lựa chọn 36

8.1 Chi phí pipeline quan sát được 36
8.2 Chọn bằng validation trước khi diễn giải test 36
8.3 Ý nghĩa đối với triển khai 38

9 Giới hạn, độ tin cậy và khả năng tái lập 38

10 Kết luận và đối chiếu đề bài 39

Tài liệu tham khảo 40

1 Bài toán và mục tiêu nghiên cứu

1.1 Phạm vi Project 2

NLBSE'23 tổ chức hai nhánh khác nhau: issue report classification và code comment classification. Nhánh code comment cung cấp 6.738 câu và 19 bộ phân loại nhị phân Random Forest gốc[1]. Vì vậy, tên Project 2, quy mô 6.738 và yêu cầu multi-label trong đề bài được hiểu nhất quán là phân loại câu chú thích mã nguồn. Báo cáo không sử dụng bộ issue report nhiều lớp.

Theo yêu cầu học phần, đề tài triển khai các bộ phân loại Deep Learning, đánh giá bằng metric phù hợp, dùng k-fold CV và so sánh với Transformer-based RoBERTa. E1 là baseline RoBERTa triển khai trong cùng dữ liệu và protocol với các phương pháp khác. E1 không phải điểm Random Forest chính thức; hai đối chứng có vai trò khác nhau.

1.2 Định nghĩa multi-label và ý nghĩa ứng dụng

Một câu comment có thể vừa mô tả cách sử dụng, vừa giải thích tham số hoặc chi tiết triển khai. Mỗi câu do đó có vector nhiều nhãn nhị phân, không bị ép chọn đúng một lớp. Đầu vào hiện tại chỉ là văn bản câu, chưa có toàn bộ class, toàn bộ comment hoặc đoạn code đi kèm. Kết quả có thể hỗ trợ tìm kiếm tài liệu và tổ chức thông tin cho người đọc mã.

Với từng ngôn ngữ ℓ, đặt

$$\mathscr{D}^{(\ell)} = \{(x_i, \boldsymbol{y}_i)\}_{i=1}^{N_\ell}, \qquad \boldsymbol{y}_i \in \{0,1\}^{C_\ell}, \quad C_\ell \in \{5,7,7\}. \tag{1}$$

Model sinh logit z_{ic}, xác suất $p_{ic} = \sigma(z_{ic})$ và $\widehat{y}_{ic} = \mathbb{I}[p_{ic} \geq t_c]$. Sigmoid theo nhãn cho phép nhiều nhãn cùng dương, không dùng softmax ép nhãn loại trừ nhau. Ngưỡng mặc định là 0.5; E4 hiệu chỉnh threshold bằng OOF validation.

1.3 Chiến lược và câu hỏi nghiên cứu

Các exp được nhóm theo thành phần thay đổi: loss xử lý imbalance, hậu xử lý threshold, biểu diễn/tương quan nhãn, regularization và pretrained backbone. Đối chứng chính là E1–E2/E3/E7 cho loss; E1–E4 cho ngưỡng; E1–E5 cho attention; E5–E6 cho contrastive; E1–E8/E10 cho regularization; E1–E11 cho backbone; E5–E12 cho graph trên attention. Mục tiêu là giải thích điều gì được cải thiện và đổi lại điều gì, không chỉ tìm một hàng có test cao nhất.

2 Phân tích khám phá dữ liệu (EDA)

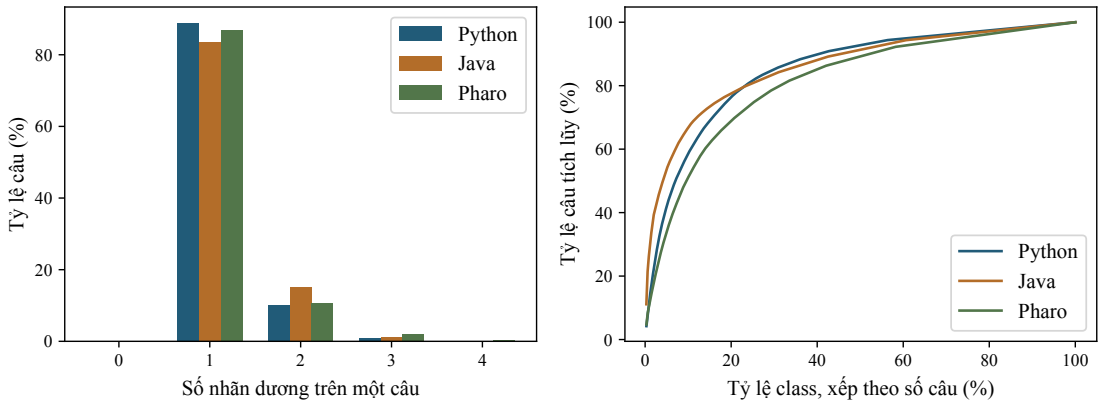
Phần EDA khảo sát 6.738 câu trong bộ chia processed trước khi trình bày quy trình tiền xử lý. Các hình được vẽ lại từ hướng phân tích trong notebook để khớp dữ liệu hiện tại. Đây là kiểm toán hậu nghiệm; test không dùng để chọn lại cấu hình. Kết quả truncation cập nhật giới hạn nêu ở Mục 9.

2.1 Đơn vị mẫu, cardinality và độ thưa nhãn

Một dòng trong EDA là một ID câu sau pivot, không phải một dòng long. Không cộng các file preprocessing như bộ dữ liệu độc lập. Label cardinality là số nhãn dương trung bình/câu; density là cardinality chia số nhãn.

Bảng E.1: Độ đa nhãn và độ thưa, tính trên toàn bộ câu.

Ngôn ngữ	Cardinality	Density	Nhiều nhãn	Tối đa
Python	1.121	22.4%	11.1%	3
Java	1.177	16.8%	16.4%	3
Pharo	1.158	16.5%	13.1%	4



Hình E.1: Trái: phân bố số nhãn dương/câu. Phải: mức tập trung câu theo class; các class được xếp giảm dần theo số câu.

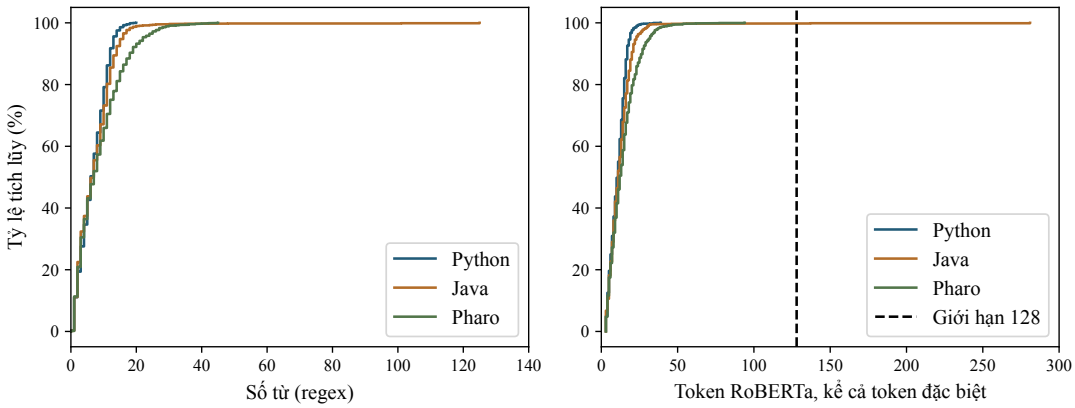
Không có câu rỗng nhãn; 11–16% câu có nhiều nhãn, nên softmax một lớp sai giả định. Density 16.5–22.4% cho thấy negative chiếm đa số, nhưng chưa chứng minh Weighted BCE hay ASL tốt hơn BCE.

2.2 Độ dài văn bản và token thực tế

Độ dài từ dùng biểu thức tách từ chữ/số; ký tự tính trên text giữ nguyên. Token được đo bằng fast tokenizer đúng backbone, có token đặc biệt, không padding và không truncation. Từ regex không tương đương subword.

Bảng E.2: Phân bố độ dài. TB: trung bình; P95: phân vị 95.

Ngôn ngữ	Đơn vị	TB	Median	P95	Max
Python	Từ	6.7	6	13.0	20
Python	Ký tự	38.7	37	72.3	103
Python	Token	10.7	11	19.0	39
Java	Từ	7.4	7	16.0	125
Java	Ký tự	45.8	43	90.0	1321
Java	Token	12.3	11	23.0	281
Pharo	Từ	8.7	7	22.0	45
Pharo	Ký tự	54.2	44	132.8	375
Pharo	Token	14.2	12	32.0	94



Hình E.2: Hàm phân phối tích lũy độ dài trên toàn bộ câu. Đường đứt biểu thị giới hạn 128 token; hình giữ cả miền ngoại lệ Java.

Pharo dài hơn ở phần lớn phân bố: P95 là 32 token, so với 19 của Python và 23 của Java. Java có đuôi ngoại lệ dài nhất, tới 281 token; chỉ nhìn trung bình sẽ bỏ qua điều này. P99 token lần lượt là 23, 31 và 43.

2.3 Giới hạn 128 token và đặc điểm text

Bảng E.3: Câu vượt giới hạn 128 token. Hai tokenizer cho cùng độ dài trên dữ liệu này.

Ngôn ngữ	Số câu	Tỷ lệ	Train	Test
Python	0	0.000%	0	0
Java	5	0.207%	3	2
Pharo	0	0.000%	0	0

Cả RoBERTa và CodeBERT chỉ cắt 5 câu Java: 3 train và 2 test, tương đương 0.207% Java và 0.074% toàn bộ dữ liệu. Không có câu Python hoặc Pharo bị cắt. Tổng phần chiều dài vượt giới hạn là 477 vị trí token; đây là hiệu số độ dài encoding, không phải 477 từ hay số thông tin ngữ nghĩa mất đi. Có token đặc biệt nên không diễn giải 128 là 128 token nội dung.

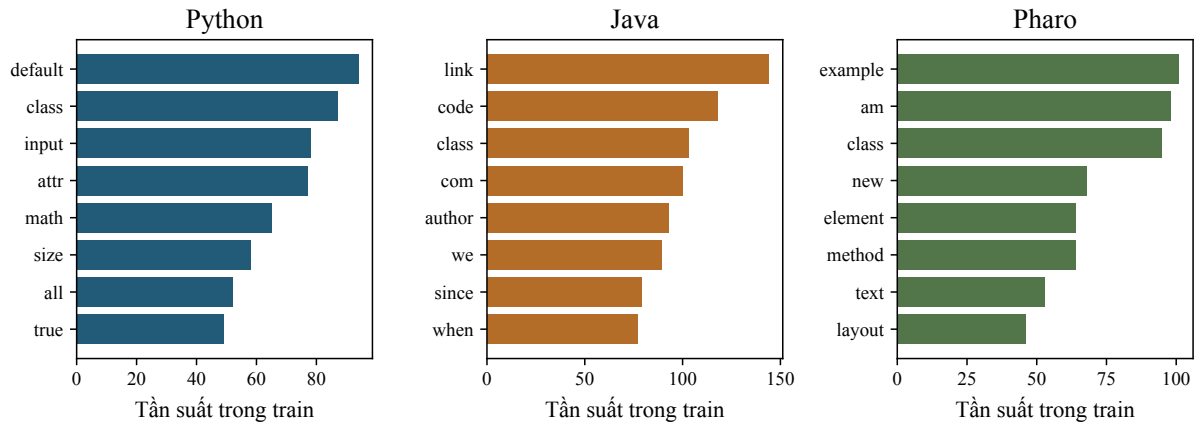
Giới hạn 128 hợp lý về độ bao phủ ở dữ liệu hiện tại. Tăng lên 256 không thể được kỳ vọng cải thiện phần lớn câu; cần thí nghiệm riêng trên validation và phân tích các ngoại lệ. Hai test bị cắt không được dùng để chọn lại max length sau khi đã xem kết quả. Cùng độ dài tokenizer cũng không có nghĩa hai pretrained backbone có cùng biểu diễn hoặc chất lượng.

Bảng E.4: Đặc điểm text; "0 từ" là không khớp regex, không phải text rỗng.

Ngôn ngữ	≤ 3 từ	Chữ thường	Dấu code	0 từ
Python	27.6%	99.7%	39.2%	8
Java	32.4%	100.0%	44.4%	5
Pharo	30.5%	99.9%	58.6%	3

Khoảng 28–32% câu rất ngắn; dấu hiệu code là heuristic từ ký hiệu hoặc từ khóa, không phải bộ nhận diện ngôn ngữ lập trình. Dữ liệu gần như đã viết thường từ nguồn, phù hợp nhận định bước chuẩn hóa local không thay đổi text. Một số text chỉ có ký hiệu vẫn hợp lệ về schema nhưng ít ngữ nghĩa; chúng cần được xem xét trong lỗi dự đoán, không tự động xóa khi chưa có tiêu chí.

2.4 Từ vựng và giới hạn ngữ cảnh



Hình E.3: Tám token thường gặp nhất trên train sau lọc danh sách stopword cơ bản của notebook. Lowercase và lọc stopword chỉ phục vụ hình EDA, không thay đầu vào training.

Token được đếm bằng quy tắc chữ cái/underscore và chữ số theo sau; tần suất là số lần xuất hiện, không phải số câu hay độ quan trọng dự đoán. Phân bố từ vựng khác nhau tạo giả thuyết về khác biệt miền, nhưng một biểu đồ tần suất không chứng minh CodeBERT phù hợp hơn hay mô hình hiểu code. Cần đọc cùng kết quả E11 theo ngôn ngữ và kiểm tra lỗi có ngữ cảnh.

Các câu ngắn, text lặp và một số text giống hệt nhưng khác nhãn gợi ý rằng đầu vào một câu có thể thiếu thông tin phân biệt. Hướng mở rộng hợp lý là bổ sung câu lân cận hoặc class context trong một thử nghiệm riêng, đồng thời chia nhóm class/project nếu muốn đánh giá khả năng chuyển miền. Không dùng class như đặc trưng rồi diễn giải kết quả hiện tại là tổng quát cho class chưa từng thấy.

Tổ hợp nhãn phổ biến. Các tổ hợp dưới đây tính trên toàn bộ dữ liệu; không gộp nhãn cùng tên giữa các ngôn ngữ thành một target chung.

Python. Parameters (716 câu); Usage (644 câu); Summary (389 câu).

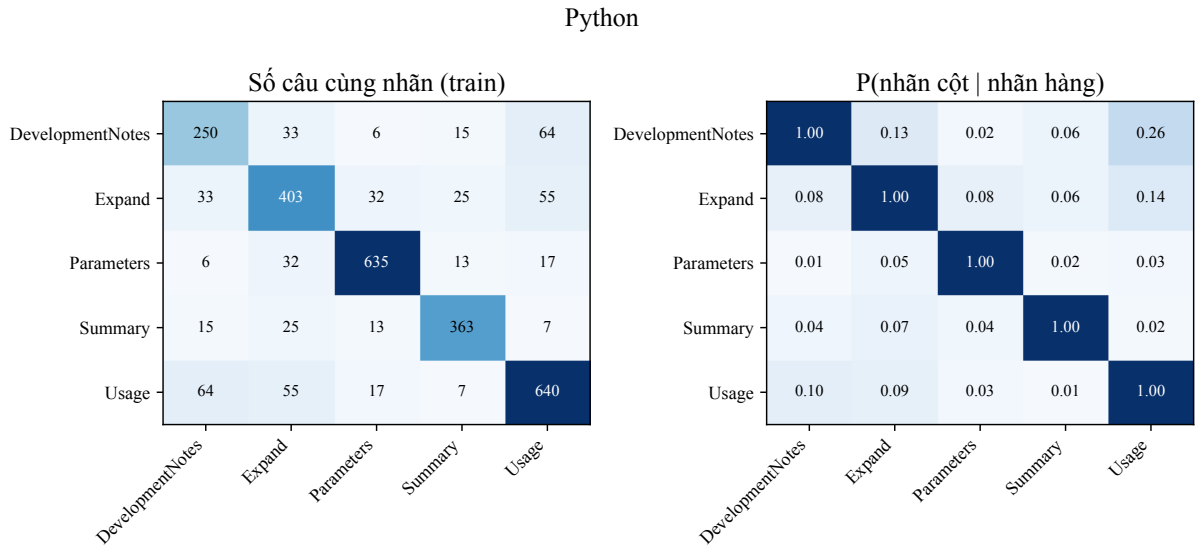
Java. usage (747 câu); Expand (362 câu); summary (314 câu).

Pharo. Example (700 câu); Keymessages (245 câu); Responsibilities (216 câu).

Tổ hợp một nhãn chiếm ưu thế không loại bỏ vai trò của quan hệ nhãn; ngược lại, nó giới hạn số mẫu hỗ trợ cho việc học cạnh graph và contrastive. Cần phân biệt số lượng tuyệt đối với độ mạnh quan hệ đã chuẩn hóa.

2.5 Quan hệ nhãn: Python

Ma trận trái tính số cùng dương trên train; đường chéo là support từng nhãn. Ma trận phải là xác suất có điều kiện theo hàng, nên bất đối xứng. Các số là tương quan thống kê, không phải quan hệ nhân quả hay suy diễn chắc chắn.



Hình E.4: Đồng xuất hiện nhãn trên train Python. Cột và hàng theo đúng thứ tự label cols; test không tham gia tạo prior graph.

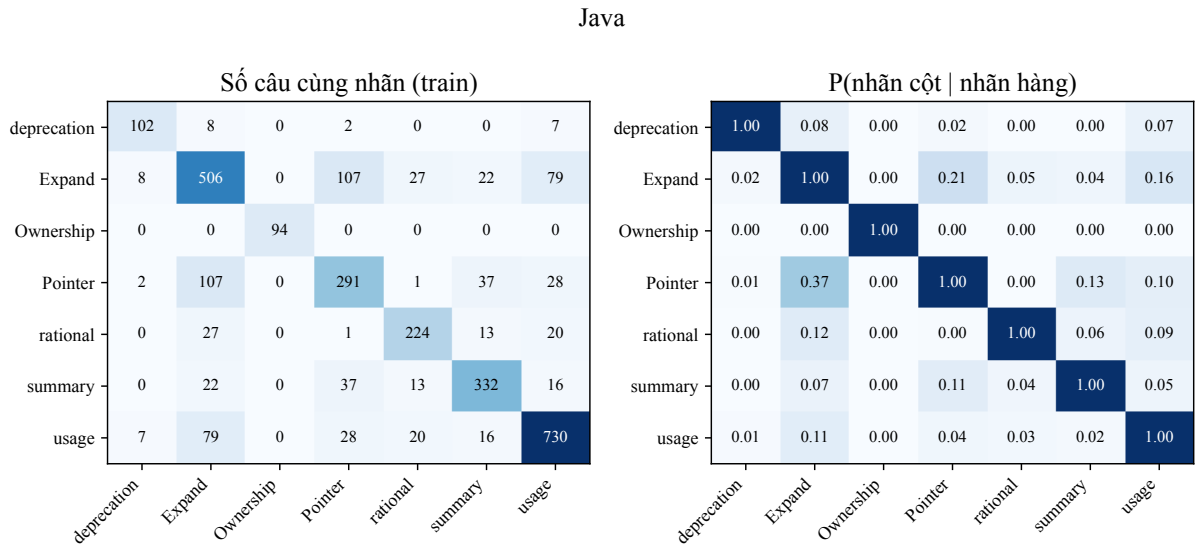
Bảng E.5: Ba cặp có số đồng xuất hiện train cao nhất; cosine dùng trong graph.

Nhãn 1	Nhãn 2	Số câu	Cosine
DevelopmentNotes	Usage	64	0.160
Expand	Usage	55	0.108
DevelopmentNotes	Expand	33	0.104

DevelopmentNotes cùng Usage xuất hiện 64 lần, nhưng chỉ 25.6% câu DevelopmentNotes mang Usage và 10.0% câu Usage mang DevelopmentNotes. Không nên biến cạnh này thành luật gán nhãn. Chỉ ba cặp vượt ngưỡng cosine 0.1 ở full train, nên prior của E9/E12 khá thưa. Tương quan ít và sentence features chưa tốt có thể hạn chế lợi ích graph; đây là giả thuyết cần ablation.

2.6 Quan hệ nhãn: Java

Ma trận trái tính số cùng dương trên train; đường chéo là support từng nhãn. Ma trận phải là xác suất có điều kiện theo hàng, nên bất đối xứng. Các số là tương quan thống kê, không phải quan hệ nhân quả hay suy diễn chắc chắn.



Hình E.5: Đồng xuất hiện nhãn trên train Java. Cột và hàng theo đúng thứ tự label cols; test không tham gia tạo prior graph.

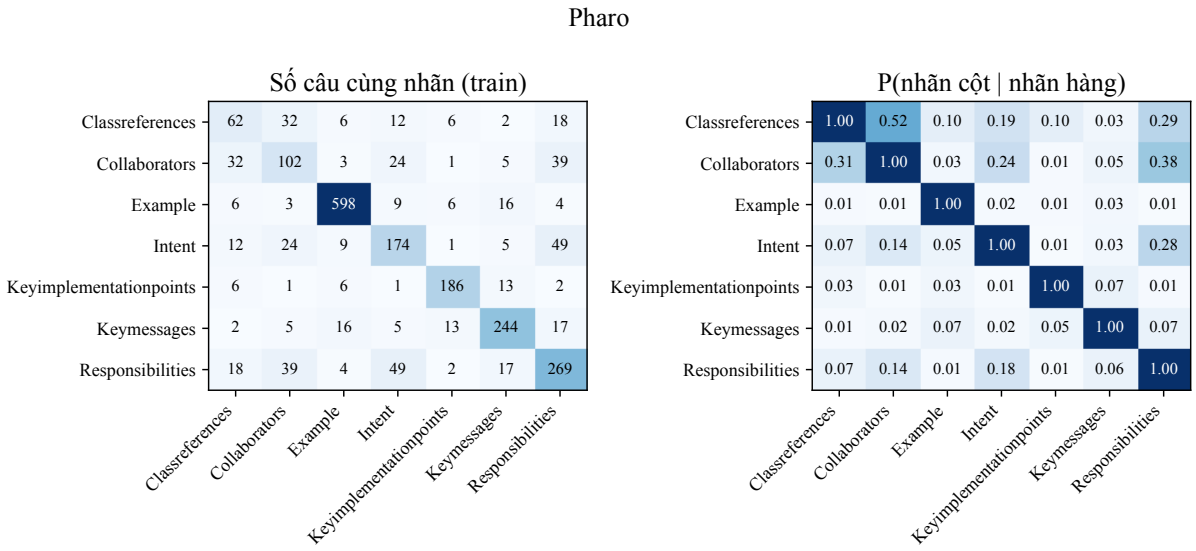
Bảng E.6: Ba cặp có số đồng xuất hiện train cao nhất; cosine dùng trong graph.

Nhãn 1	Nhãn 2	Số câu	Cosine
Expand	Pointer	107	0.279
Expand	usage	79	0.130
Pointer	summary	37	0.119

Expand–Pointer có 107 câu cùng dương, cosine 0.279. Tuy nhiên, P(Pointer | Expand) chỉ 0.211 trong khi P(Expand | Pointer) là 0.368. Prior đối xứng của GCN không giữ nguyên hai hướng điều kiện này. Việc E9 không vượt E1 rõ rệt trên Java phù hợp với cảnh báo rằng đồng xuất hiện không đủ để bảo đảm cải thiện; không thể quy nguyên nhân chỉ cho graph.

2.7 Quan hệ nhãn: Pharo

Ma trận trái tính số cùng dương trên train; đường chéo là support từng nhãn. Ma trận phải là xác suất có điều kiện theo hàng, nên bất đối xứng. Các số là tương quan thống kê, không phải quan hệ nhân quả hay suy diễn chắc chắn.



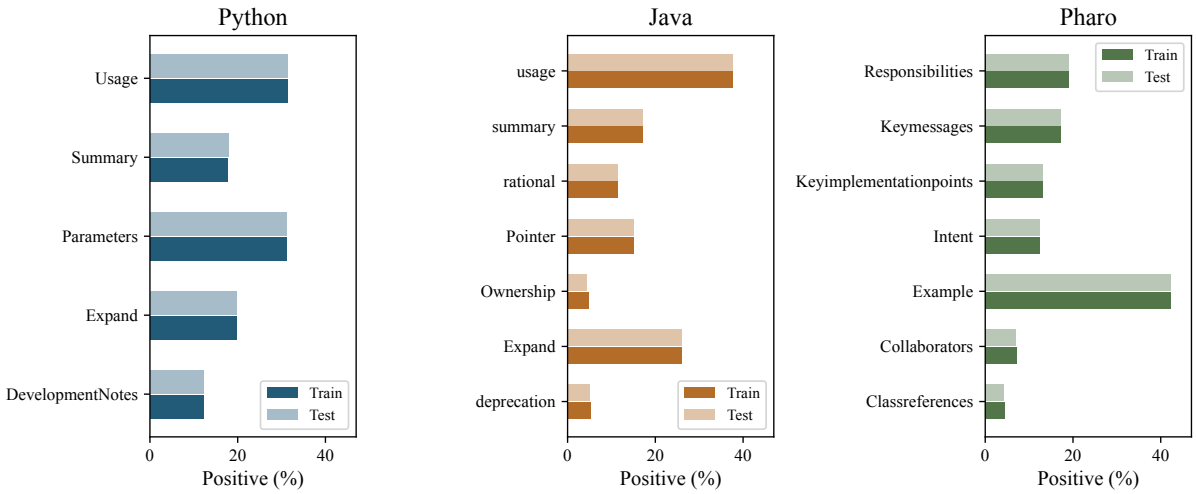
Hình E.6: Đồng xuất hiện nhãn trên train Pharo. Cột và hàng theo đúng thứ tự label cols; test không tham gia tạo prior graph.

Bảng E.7: Ba cặp có số đồng xuất hiện train cao nhất; cosine dùng trong graph.

Nhãn 1	Nhãn 2	Số câu	Cosine
Intent	Responsibilities	49	0.226
Collaborators	Responsibilities	39	0.235
Classreferences	Collaborators	32	0.402

Classreferences–Collaborators có 32 câu cùng dương, ít hơn 49 câu của Intent–Responsibilities, nhưng cosine cao hơn: 0.402 so với 0.226. P(Collaborators | Classreferences) khoảng 0.516; chiều ngược lại 0.314. Đếm thô sẽ bỏ qua quan hệ tương đối mạnh của nhãn hiếm. Graph full train có sáu cạnh vượt 0.1, vẫn chưa phải cấu trúc đầy đủ của ý nghĩa nhãn.

2.8 Phân bố train/test



Hình E.7: Tỷ lệ positive trên train và test của bộ chia processed hiện tại; không dùng partition nhị phân gốc. Hai cột gần nhau chỉ xác nhận cân bằng marginal theo nhãn.

Bảng E.8: Độ lệch prevalence train/test lớn nhất, đơn vị điểm phần trăm.

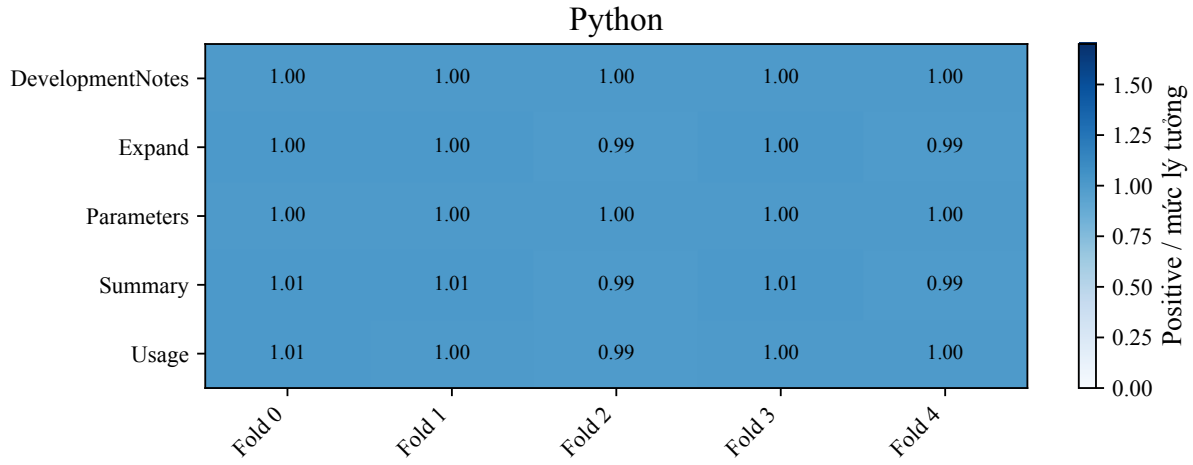
Ngôn ngữ	Nhãn lệch nhất	Độ lệch tuyệt đối
Python	DevelopmentNotes	0.098
Java	Ownership	0.522
Pharo	Keyimplementationpoints	0.188

Độ lệch marginal nhỏ: Python dưới 0.10, Java dưới 0.53 và Pharo dưới 0.19 điểm phần trăm. Vì vậy chênh lệch lớn giữa CV và test không thể được giải thích chỉ bằng tỷ lệ positive toàn cục. Phân bố class, độ khó câu, tổ hợp nhãn và cỡ mẫu vẫn có thể khác; chưa có kiểm định dịch chuyển ngữ nghĩa hoặc near-duplicate đầy đủ.

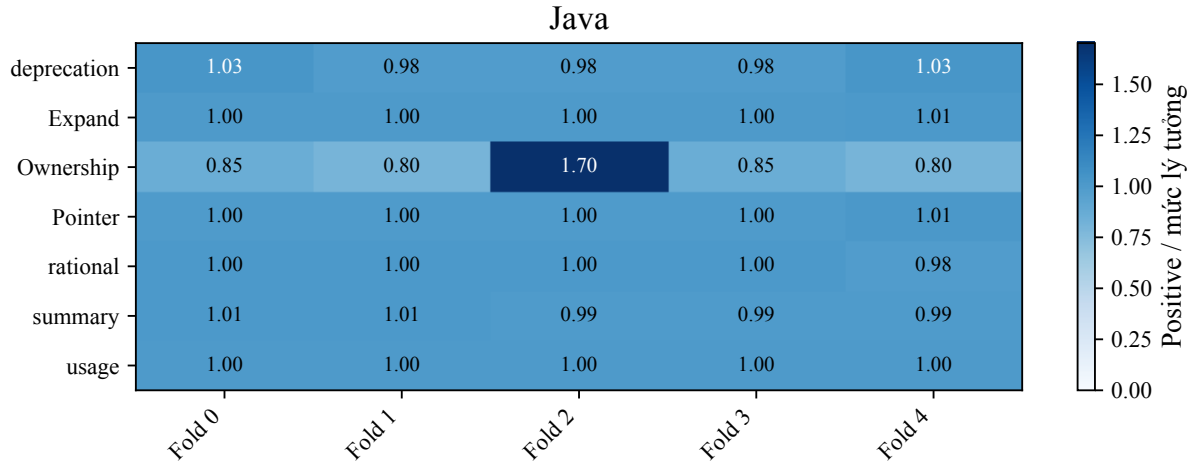
Các hình này là kiểm toán sau thí nghiệm. Không chọn lại test split để tạo metric thuận lợi hơn. Một holdout mới hoặc grouped/nested evaluation là bước xác nhận phù hợp hơn việc tiếp tục tối ưu trên test đã quan sát.

2.9 Cân bằng positive của năm fold

Với nhãn c, giá trị trong hình bằng số positive của validation fold chia cho tổng positive train/5. Mức 1.00 là lý tưởng về số positive; đây không phải metric dự đoán. Mỗi mẫu vẫn làm validation đúng một lần.

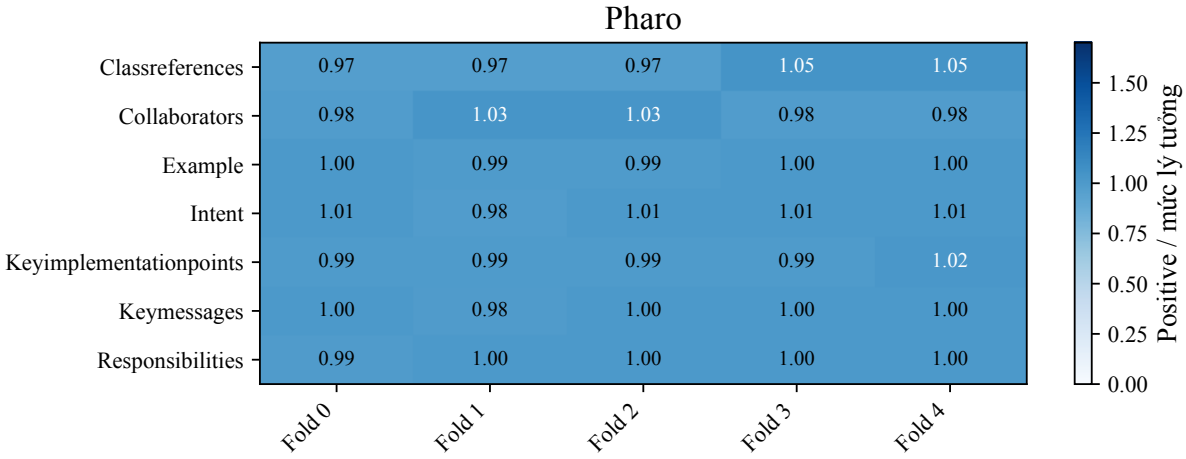


Hình E.8: Python: tỷ lệ positive so với mức lý tưởng trong từng fold.



Hình E.9: Java: Ownership có một fold khoảng 1.70; nhóm text phải giữ nguyên dù nhãn mất cân bằng hơn.

Python nằm trong khoảng 0.992–1.008, trong khi Java là 0.798–1.702. Khác biệt SD F1 giữa các ngôn ngữ vì vậy cần được đọc cùng độ khó và phân bố fold; chưa thể xem SD cao chỉ là model bất ổn.



Hình E.10: Pharo: tỷ lệ positive theo fold nằm trong khoảng 0.968–1.048.

2.10 Text trùng, tập trung class và leakage

Bảng E.9: Kiểm toán toàn bộ dữ liệu. Trùng: số câu trong nhóm text lặp; khác: số nhóm có vector nhãn khác; câu khác: số câu trong các nhóm đó.

Ngôn ngữ	Trùng	Khác	Câu khác	Class	Top 10
Python	368	16	74	330	30.6%
Java	1002	13	200	348	44.1%
Pharo	205	5	18	328	24.1%

Java có 1.002/2.418 câu nằm trong nhóm text lặp, và 200 câu thuộc 13 nhóm có nhiều vector nhãn. Dữ liệu lặp không nhất thiết sai: text có thể đến từ class khác nhau. Giữ toàn bộ nhưng ràng buộc cùng group tránh trùng chính xác giữa train/test và giữa hai phía mỗi fold; kiểm toán cho kết quả giao group bằng 0 ở cả ba ngôn ngữ.

Top 10 class chiếm 44.1% câu Java, so với 30.6% Python và 24.1% Pharo. Class lớn nhất lần lượt có 269, 108 và 84 câu. Kết quả chịu ảnh hưởng không đều từ các nguồn class; class giao train/test như Mục 3 nên chưa đại diện cho project mới. Lowercase thêm trong kiểm tra không làm đổi số câu thuộc nhóm text trùng; điều đó không loại trừ near-duplicate.

2.11 EDA và chiến lược thử nghiệm

Bảng E.10: Quan sát, ý nghĩa và hành động kiểm chứng.

Quan sát	Ý nghĩa	Chiến lược
11–16% câu nhiều nhãn	Nhãn không loại trừ	Sigmoid/BCE; đọc micro và macro.
Density 16.5–22.4%	Nhiều negative	E2/E3/E7; so cả precision, recall và HL.
P95 chỉ 19–32 token	Phần lớn câu ngắn	Giữ 128 làm cấu hình chung; đo chi phí trước khi tăng.
Chỉ 5 câu Java bị cắt	Lợi ích tăng length có thể cục bộ	Kiểm tra validation ngoại lệ; không chỉnh theo test.
Graph thưa, quan hệ có hướng	Đồng xuất hiện không phải luật nhãn	E9/E12; ablation prior, ngưỡng và residual.
Text trùng nhiều, class tập trung	Rủi ro leakage và thiên lệch nguồn	Group text; bổ sung đánh giá class/project mới.
Marginal train/test gần nhau	Cân bằng nhãn chưa đủ bảo đảm độ khó	Đọc fold, class, per-label và lặp seed.
Nhiều câu rất ngắn	Có thể thiếu ngữ cảnh	Thử context ở một protocol mới.

EDA cung cấp căn cứ thiết kế và giới hạn diễn giải, không tự chứng minh một phương pháp tốt hơn. Các kết quả ở Mục 6–8 vẫn là bằng chứng thực nghiệm: loss đổi đánh đổi FP/FN; graph không thắng đồng nhất; CodeBERT phụ thuộc miền; và ứng viên CV không nhất thiết dẫn đầu test.

Phần bổ sung này không thay train/test, folds, model hay metric của 36 run đã báo cáo. Các hướng tăng độ dài, thêm context hoặc thay graph là thử nghiệm tương lai và cần chọn bằng validation theo quy tắc công bố trước.

3 Dữ liệu và quy trình tiền xử lý

3.1 Nguồn, cấu trúc và mất cân bằng

Dữ liệu local được đối chiếu với repository code comment NLBSE'23[2]. Raw gồm ID câu, class, text, category, nhãn instance type và partition. Một ID xuất hiện theo từng category nên số dòng raw lớn hơn số câu. Category xác định nhãn đang xét; instance type mới xác định positive/negative.

Bảng 1: Quy mô và phân vùng dữ liệu. Dấu phẩy phân tách hàng nghìn.

Ngôn ngữ	Dòng raw	Số câu	Nhãn	Train	Test
Python	12,775	2,555	5	2,044	511
Java	16,926	2,418	7	1,934	484
Pharo	12,355	1,765	7	1,411	354

Tổng số câu là $2.555 + 2.418 + 1.765 = 6.738$. Số nhãn dương trung bình mỗi câu lần lượt xấp xỉ 1.12, 1.18 và 1.16. Một câu được đếm ở nhiều nhãn, nên tổng tỷ lệ positive không nhất thiết bằng 100%. Các hệ nhãn được giữ riêng; không xem 19 tên nhãn là 19 target được quan sát đầy đủ trên mọi câu.

Bảng 2: Số câu positive theo nhãn và phân vùng đã cố định.

Ngôn ngữ	Nhãn	Tổng	Train	Test	Tỷ lệ (%)
Python	DevelopmentNotes	312	250	62	12.2
Python	Expand	504	403	101	19.7
Python	Parameters	794	635	159	31.1
Python	Summary	454	363	91	17.8
Python	Usage	800	640	160	31.3
Java	deprecation	127	102	25	5.3
Java	Expand	632	506	126	26.1
Java	Ownership	115	94	21	4.8
Java	Pointer	364	291	73	15.1
Java	rational	280	224	56	11.6
Java	summary	415	332	83	17.2
Java	usage	912	730	182	37.7

Ngôn ngữ	Nhãn	Tổng	Train	Test	Tỷ lệ (%)
Pharo	Classreferences	77	62	15	4.4
Pharo	Collaborators	127	102	25	7.2
Pharo	Example	748	598	150	42.4
Pharo	Intent	218	174	44	12.4
Pharo	Keyimplementationpoints	232	186	46	13.1
Pharo	Keymessages	305	244	61	17.3
Pharo	Responsibilities	336	269	67	19.0

Python có nhãn thấp nhất khoảng 12.2%, trong khi Java và Pharo có nhãn chỉ 4–5%. Example của Pharo chiếm 42.4%, còn Classreferences chỉ 4.4%. Đây là cơ sở khảo sát loss và threshold theo nhãn; micro F1 đơn độc có thể che khuất các nhãn ít mẫu.

3.2 Kiểm tra raw, pivot và chuẩn hóa

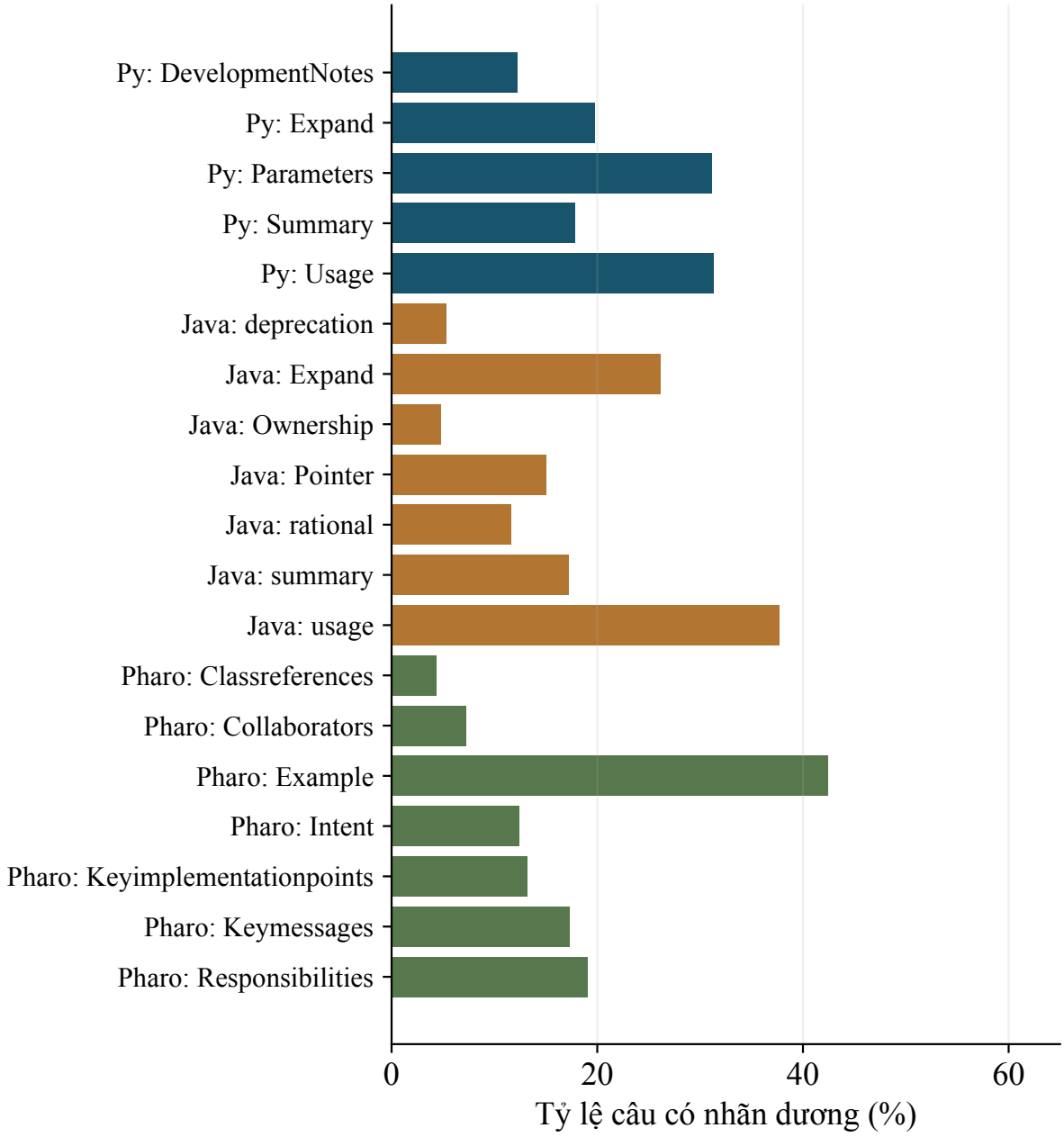
Pipeline kiểm tra text/class rỗng, nhãn ngoài 0/1, category lạ, cặp ID–category trùng, cùng ID nhưng khác text/class và ID thiếu category. Các mục lỗi đều bằng 0 trên dữ liệu hiện tại. Sau pivot, mỗi ID một dòng và mỗi nhãn một cột 0/1; tổng positive được đối soát với raw.

Text chỉ chuẩn hóa khoảng trắng và bỏ khoảng trắng hai đầu, không bổ sung lowercase, stemming, bỏ stopword hoặc xóa ký hiệu code. Ở bản đang dùng, bước này không làm đổi text nào. Raw là đầu vào của pipeline đề tài, không có nghĩa comment nguyên bản chưa từng được bên cung cấp xử lý.

3.3 Nhóm text trùng và giới hạn ngữ cảnh

Các ID khác nhau có thể có cùng text. Những câu giống hệt sau chuẩn hóa được gán cùng group và không được tách sang train/test hoặc train/validation. Không tự xóa câu trùng nhưng khác vector nhãn vì có thể chúng xuất hiện trong những class khác nhau.

Java có 1.002 câu nằm trong nhóm trùng text và nhóm lớn nhất 51 câu. Chia theo ID đơn thuần có thể rò rỉ nội dung dù không trùng ID. Grouping ngăn trùng chính xác, nhưng chưa loại near-duplicate hoặc mẫu diễn đạt tương tự. Class vẫn giao nhau, nên kết quả không đại diện cho class/project hoàn toàn mới. Câu cùng text nhưng khác nhãn còn cho thấy giới hạn của model chỉ nhận một câu mà không có ngữ cảnh.



Hình 1: Phân bố positive của ba hệ nhãn. Đây là thống kê dữ liệu, không phải kết quả dự đoán.

Bảng 3: Nhóm câu. Khác nhãn: nhóm có nhiều vector nhãn; class chung: class có ở cả train/test.

Ngôn ngữ	Nhóm	Trùng	Lớn nhất	Khác nhãn	Class chung
Python	2299	112	19	16	128
Java	1668	252	51	13	110
Pharo	1620	60	18	5	120

3.4 Bộ chia chung và 5-fold cross-validation

Mỗi nhóm được biểu diễn bằng số positive từng nhãn và số câu. Thuật toán greedy xét nhóm lớn trước, gán vào phần làm tăng ít nhất độ lệch chuẩn hóa so với phân bố lý tưởng, với seed 42. Từ năm phần gần cân bằng, chọn một phần lệch positive nhỏ nhất làm test khoảng 20%, không dựa vào model score. Trên train còn lại, tạo 5 fold với ràng buộc nhóm tương tự.

Đây là bộ chia nhóm đa nhãn tự xây dựng, không phải gọi trực tiếp một hàm stratified k-fold thông thường. Mọi exp đọc cùng thứ tự train/test, nhãn và indices đã lưu. Mỗi mẫu train làm validation đúng một lần và thuộc fold train bốn lần.

Bảng 4: Số mẫu validation của các fold đã lưu, chỉ số bắt đầu từ 0.

Ngôn ngữ	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4
Python	409	409	408	409	409
Java	386	386	388	387	387
Pharo	283	282	282	282	282

Giữ group cùng phía được ưu tiên hơn cân bằng tuyệt đối. Một nhóm 32 câu Ownership của Java làm số positive ở một validation fold khoảng 1.70 lần mức lý tưởng. Đây là lý do phải đọc SD fold và metric theo nhãn, không tách group để làm đẹp phân bố.

Bộ chia này thay partition theo từng category của nguồn. Nó phù hợp so sánh E1–E12 và yêu cầu k-fold, nhưng không cho phép tuyên bố vượt Random Forest chính thức dưới một test split khác.

4 Mô hình và ý nghĩa các thử nghiệm

4.1 Backbone và baseline chung

Backbone chính là RoBERTa-base[3], hidden size 768, fine-tune toàn bộ. Tokenizer tương ứng padding/truncation đến 128 token. Trạng thái token đầu câu được dùng như CLS. Tất cả exp chỉ nhận comment, kể cả CodeBERT; không tự thêm code hoặc metadata class vào đầu vào.

Bảng 5: Chiến lược E1–E12. Mỗi exp chạy trên ba ngôn ngữ.

Exp	Phương pháp	Câu hỏi và đối chứng
E1	RoBERTa + BCE	Baseline Transformer cùng protocol.
E2	Weighted BCE	Trọng số positive có giúp nhãn hiếm? So với E1.
E3	Focal Loss	Tập trung ví dụ khó có tăng chất lượng? So với E1.
E4	Threshold Tuning	Đổi ngưỡng có cải thiện quyết định? So với E1.
E5	Label-Aware Attention	Context theo nhãn có hơn CLS? So với E1.
E6	Attention + Contrastive	Ràng buộc câu–nhãn có ích thêm? So với E5.
E7	Asymmetric Loss	Giảm easy negatives có cân bằng tốt? So với E1/E2/E3.
E8	R-Drop	Nhất quán giữa hai dropout forward có ích? So với E1.
E9	Label GNN v2	Tương quan nhãn trên hệ nhãn nhỏ có ích? So với E1.
E10	Adversarial FGM	Nhiễu embedding có tăng tổng quát? So với E1.
E11	CodeBERT + BCE	Backbone pretrain code có phù hợp? So với E1.
E12	Attention + GNN	Graph trên context có bổ sung cho E5?

4.2 Loss: E1, E2, E3 và E7

E1 dùng đầu tuyến tính $\boldsymbol{z}_i = W\boldsymbol{h}_i + \boldsymbol{b}$ và BCE:

$$\mathscr{L}_{BCE} = -\frac{1}{|B|C}\sum_{i,c}[y_{ic}\log p_{ic} + (1-y_{ic})\log(1-p_{ic})]. \tag{2}$$

Loss được tính từ logits để ổn định số học. E2 nhân thành phần positive với $w_c = n_c^- / n_c^+$, chỉ tính từ fold train; nhãn không có positive hoặc negative dùng weight 1. Model cuối tính lại từ full train.

E3 dùng sigmoid focal loss $-\alpha_t(1-p_t)^\gamma \log p_t$ với $\gamma = 2$ và không alpha weighting. Nó giảm đóng góp các cặp câu–nhãn dễ[4]. E7 áp dụng ASL[5], đặt $q = \min(1, 1-p+0.05)$:

$$\mathscr{L}_{ASL} = -y(1-p)\log p - (1-y)(1-q)^4 \log q. \tag{3}$$

Hệ số focusing được detach, loss lấy trung bình như các exp khác. E2 đổi chi phí positive, E3 tập trung ví dụ khó, E7 tác động positive và negative bất đối xứng. Chúng không được kỳ vọng tối ưu đồng thời mọi metric chỉ vì dataset có imbalance.

4.3 Ngưỡng quyết định: E4

E4 giữ kiến trúc và BCE như E1. Xác suất validation từ best epoch từng fold được ghép thành OOF đúng thứ tự train. Mỗi nhãn chọn threshold tối đa F1 trên lưới $\{0.05, 0.10, \ldots, 0.95\}$. Hòa điểm ưu tiên gần 0.5, sau đó ưu tiên ngưỡng nhỏ hơn. Test không tham gia tuning. Metric CV E4 vẫn ở ngưỡng 0.5 nên giống E1. F1 trên chính OOF đã tuning không là đánh giá độc lập.

E4 thay quyết định nhị phân; nếu model không đổi thì thứ hạng xác suất và AP không đổi. Vì tối ưu riêng từng nhãn, phương pháp không trực tiếp tối ưu micro F1 hoặc Hamming loss.

4.4 Attention, contrastive và GNN: E5, E6, E9, E12

E5 học query q_c cho mỗi nhãn và attention trên token không padding:

$$a_{ict} = \mathrm{softmax}_t(q_c^\top \tanh(W_a h_{it} + b_a)/\sqrt{768}), \quad r_{ic} = \sum_t a_{ict} h_{it}. \tag{4}$$

Logit nhãn được tạo từ context r_{ic} riêng. Query là tham số học, không phải embedding có sẵn từ mô tả tên nhãn. E6 thêm sentence–label contrastive loss: cosine giữa CLS đã chiếu và query, temperature 0.1, target đều trên nhãn positive. Loss bằng BCE cộng 0.1 lần term phụ. Đầu ra multi-label vẫn là sigmoid.

E9 dùng GCN nhãn, lấy cảm hứng từ hướng label correlation[7]. Chỉ từ fold train, tính $M = Y^\top Y$ và $A_{cd} = M_{cd}/\sqrt{s_c s_d}$. Bỏ cạnh ngoài đường chéo dưới 0.1, thêm self-loop và chuẩn hóa đối xứng thành $\widehat{A}$. Với embedding nhãn E:

$$H = E + \mathrm{Linear}_2(\widehat{A}\,\mathrm{ReLU}(\mathrm{Linear}_1(\widehat{A}E))). \tag{5}$$

GCN hidden size 128, đầu ra 768. E9 v2 khởi tạo embedding nhãn và bias từ đầu CLS

như E1; lớp cuối nhánh GCN bằng 0 để ban đầu tương đương baseline và graph học phần hiệu chỉnh. Logit phân loại không chia cho $\sqrt{768}$; tránh shared bias ngẫu nhiên chi phối mọi nhãn.

Graph có 5/7/7 node theo ngôn ngữ, tính lại trên full train cho model cuối. Không tạo graph từ toàn bộ train trước CV hoặc từ test. E12 dùng graph tương tự trên context attention theo từng câu, tối ưu BCE, không tự cộng contrastive loss của E6. Graph chỉ

Bảng 6: Graph full train ở ngưỡng 0.1. Số cạnh không đếm self-loop.

Ngôn ngữ	Node	Cạnh	Node không nối nhãn khác
Python	5	3	2
Java	7	3	3
Pharo	7	6	3

mã hóa đồng xuất hiện positive, không phải mọi quan hệ ngữ nghĩa. Node không nối nhãn khác vẫn có self-loop và residual. E9–E12 thay cả node features và khởi tạo nên không là ablation chỉ thay một thành phần.

4.5 Regularization và backbone: E8, E10, E11

E8 dùng R-Drop[6]: hai forward dropout cho cùng câu, lấy trung bình BCE và cộng KL đối xứng giữa các phân phối Bernoulli theo nhãn, hệ số 1. Không softmax trên nhãn. Evaluation dùng một forward.

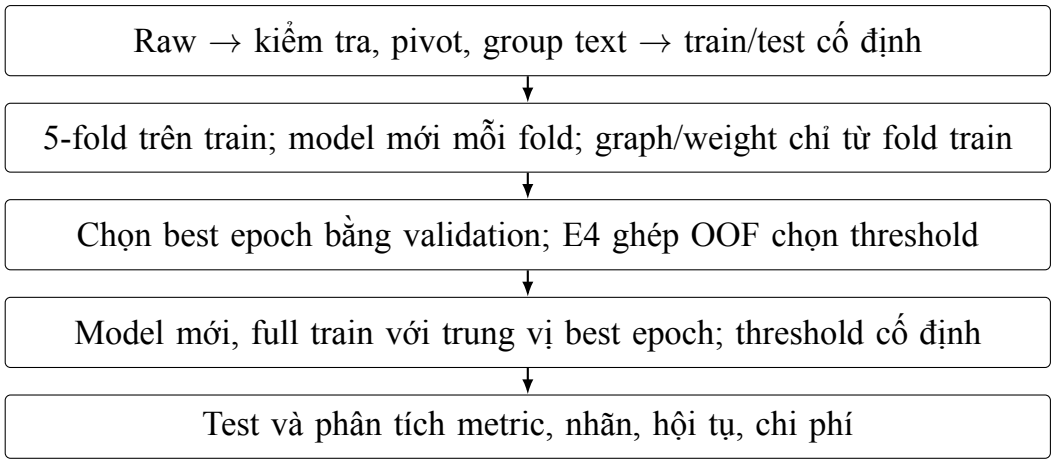
E10 dùng FGM ở input embedding[8]. Với gradient loss sạch g_i, nhiễu từng câu là $r_i = g_i / \max(\|g_i\|_2, 10^{-12})$, tức $\varepsilon = 1$. Padding không nhận nhiễu. Loss là trung bình BCE sạch và BCE đối kháng. Nhiễu detach, không sửa trực tiếp tham số embedding. Đây là FGM một bước, không phải PGD hay virtual adversarial training.

E11 thay bằng CodeBERT-base, giữ CLS, BCE và cùng đầu vào comment. CodeBERT được pretrain với code và ngôn ngữ tự nhiên[9]. Nguồn pretrain có Python/Java nhưng không có Pharo[10]. Đây là cơ sở cho giả thuyết phù hợp miền, chưa chứng minh pretraining là nguyên nhân duy nhất của các chênh lệch quan sát.

5 Thiết kế đánh giá và lựa chọn model

5.1 Pipeline và cấu hình

Mỗi fold khởi tạo lại từ pretrained. Seed cơ sở 42, seed fold bằng 42 cộng chỉ số fold; model cuối dùng 42. AdamW có LR 2×10^{-5}, weight decay 0.01, linear scheduler,



Hình 2: Pipeline chung. Các bước dựa vào nhãn để tạo model không đọc nhãn test.

warmup 10%, clipping norm 1. Batch train/evaluation 16/32, gradient accumulation 1, mixed precision tắt, DataLoader workers 0. Máy có hai A30 24 GB; mỗi model trên một GPU, các exp độc lập có thể chạy song song.

Ngân sách tối đa 10 epoch/fold. Mặc định chọn checkpoint theo validation micro F1, patience 2, hòa giữ epoch trước. E9 v2 dùng patience 3, tối thiểu 5 epoch trước khi cho phép dừng, và micro AP phân xử khi F1 bằng nhau. Đây là khác biệt phải ghi nhận khi diễn giải E9.

Sau CV, model mới train full train với trung vị best epoch của năm fold, không dùng test chọn checkpoint. Minimum epoch của E9 điều khiển độ dài tìm kiếm, không ép best epoch/model cuối tối thiểu 5. Mỗi exp/ngôn ngữ có 5 lượt CV và 1 full train; 36 run tương ứng 216 lượt fine-tune. Thời gian là toàn pipeline, không chỉ một model.

5.2 Metric phù hợp

Micro F1 cộng số đếm mọi nhãn; macro F1 cho mỗi nhãn trọng số bằng nhau:

$$F_1^{micro} = \frac{2\sum_c TP_c}{2\sum_c TP_c + \sum_c FP_c + \sum_c FN_c}, \quad F_1^{macro} = \frac{1}{C}\sum_c \frac{2TP_c}{2TP_c + FP_c + FN_c}. \tag{6}$$

Precision/recall giải thích thay đổi F1. AP dùng thứ hạng xác suất; macro AP lấy trung bình các nhãn có positive, ghi nhận không xác định nếu không có positive. Hamming loss là tỷ lệ quyết định nhãn sai:

$$HL = \frac{1}{NC}\sum_{i,c} \mathbb{I}[\widehat{y}_{ic} \neq y_{ic}]. \tag{7}$$

F1/AP càng cao càng tốt; HL càng thấp càng tốt. Không chọn bằng accuracy vì nhiều target là negative. Đối chứng dự đoán toàn âm trên test Java có HL khoảng 0.1671 nhưng F1 bằng 0: HL thấp riêng lẻ không bảo đảm nhận diện được nhãn dương.

CV báo mean và sample SD tại best epoch riêng của từng fold. SD này không là khoảng tin cậy và không mô tả biến thiên nhiều seed. CV chọn epoch trên chính validation nên mang vai trò screening model.

5.3 Quy tắc lựa chọn và kiểm tra triển khai

Tiêu chí chính là CV micro F1; hòa ưu tiên model đơn giản nếu chưa khai báo mục tiêu phụ. Không chọn lại chỉ vì test cao nhất. Nếu cần một kỹ thuật chung, dùng trung bình đều ba CV micro F1; đây không phải pooled F1 của một model 19 nhãn.

Mỗi cặp exp–ngôn ngữ giữ một run hoàn chỉnh, cùng fingerprint, thứ tự nhãn và fold indices. Các bản dở dang và E9 head cũ đã loại. Cả 36 checkpoint nạp được; test metrics nhất quán với số đếm và metric theo nhãn. Tiền xử lý tái lập trong bộ nhớ khớp chính xác dữ liệu và folds. Bộ 26 tests kiểm tra đường chạy 12 exp, gradient, graph không đọc nhãn ngoài train, khôi phục best weights và phân xử checkpoint. Kiểm tra phần mềm không chứng minh model đạt tối ưu.

6 Kết quả so sánh với baseline RoBERTa

Chữ đậm chỉ giá trị tốt nhất từng cột, không phải model được chọn bằng test. CV là mean $\pm$ SD, AP là macro AP, E9 dùng head v2.

6.1 Python: lợi ích CV không đồng nghĩa lợi ích test

CodeBERT E11 đạt CV 0.6410 ± 0.0047 so với E1 là 0.5747 ± 0.0273, hơn ở cả 5 fold. Nhưng test micro F1 chỉ đổi 0.6466 thành 0.6467. Không diễn giải gain CV 6.63 điểm phần trăm thành gain test tương tự. Test baseline cũng có thể thuận lợi hơn những validation fold hiện tại.

E9 v2 đạt test micro F1 cao nhất 0.6591 nhưng CV không đứng đầu. E7 có test macro F1 0.5885, E12 có macro AP 0.6763 cao nhất. Model tốt ở quyết định nhị phân không nhất thiết tốt nhất theo macro hoặc thứ hạng xác suất.

Bảng 7: Kết quả Python, test 511 câu.

Exp	CV micro F1 mean ± SD	Test micro F1	Test macro F1	Test macro AP	HL
E1	0.5747 ± 0.0273	0.6466	0.5755	0.6624	0.1472
E2	0.5841 ± 0.0257	0.6039	0.5633	0.6407	0.1910
E3	0.5885 ± 0.0263	0.6149	0.5381	0.6352	0.1554
E4	0.5747 ± 0.0273	0.6196	0.5838	0.6624	0.1793
E5	0.5906 ± 0.0307	0.6217	0.5279	0.6476	0.1558
E6	0.5894 ± 0.0213	0.6491	0.5781	0.6626	0.1456
E7	0.5750 ± 0.0232	0.6187	**0.5885**	0.6755	0.2137
E8	0.5841 ± 0.0315	0.6539	0.5746	0.6653	**0.1413**
E9	0.5874 ± 0.0290	**0.6591**	0.5867	0.6688	**0.1413**
E10	0.5901 ± 0.0256	0.6337	0.5537	0.6487	0.1479
E11	**0.6410** ± 0.0047	0.6467	0.5627	0.6655	0.1479
E12	0.5957 ± 0.0340	0.6399	0.5701	**0.6763**	0.1542

Bảng 8: Kết quả Java, test 484 câu.

Exp	CV micro F1 mean ± SD	Test micro F1	Test macro F1	Test macro AP	HL
E1	0.5908 ± 0.0437	0.5679	0.5997	0.6550	0.1361
E2	0.5997 ± 0.0565	0.5811	0.6339	0.6223	0.1744
E3	0.6001 ± 0.0488	0.5143	0.5358	0.6242	0.1555
E4	0.5908 ± 0.0437	0.5650	0.6354	0.6550	0.1936
E5	0.5957 ± 0.0489	0.5800	0.6165	0.6441	0.1316
E6	0.5977 ± 0.0459	0.5571	0.6022	0.6559	0.1432
E7	0.5809 ± 0.0472	0.5565	0.5931	0.6345	0.2004
E8	**0.6029** ± 0.0680	0.5682	0.5647	0.6408	0.1346
E9	0.5773 ± 0.0698	0.5698	0.5861	0.6640	0.1319
E10	0.5830 ± 0.0638	0.5290	0.5355	0.6529	0.1367
E11	0.6006 ± 0.0501	**0.6110**	**0.6778**	**0.6937**	**0.1278**
E12	0.5742 ± 0.0549	0.5735	0.6024	0.6358	0.1414

Bảng 9: Kết quả Pharo, test 354 câu.

Exp	CV micro F1 mean ± SD	Test micro F1	Test macro F1	Test macro AP	HL
E1	0.6832 ± 0.0403	0.6970	0.5777	0.6307	0.0940
E2	0.6800 ± 0.0307	0.6754	0.6171	0.6579	0.1207
E3	**0.6923** ± 0.0347	**0.7028**	0.6197	0.6707	**0.0928**
E4	0.6832 ± 0.0403	0.6895	0.6239	0.6307	0.1061
E5	0.6787 ± 0.0331	0.6711	0.5876	0.6729	0.1009
E6	0.6776 ± 0.0432	0.6813	0.5915	0.6622	0.0993
E7	0.6547 ± 0.0300	0.6686	**0.6250**	**0.6934**	0.1360
E8	0.6832 ± 0.0385	0.6829	0.5346	0.6562	0.0944
E9	0.6878 ± 0.0438	0.6813	0.5728	0.6509	0.0997
E10	0.6787 ± 0.0356	0.6844	0.5659	0.6535	0.0960
E11	0.6656 ± 0.0552	0.6622	0.5431	0.6249	0.1013
E12	0.6893 ± 0.0263	0.6879	0.6150	0.6762	0.0989

6.2 Java: CodeBERT nổi bật trên test, R-Drop dẫn nhẹ ở CV

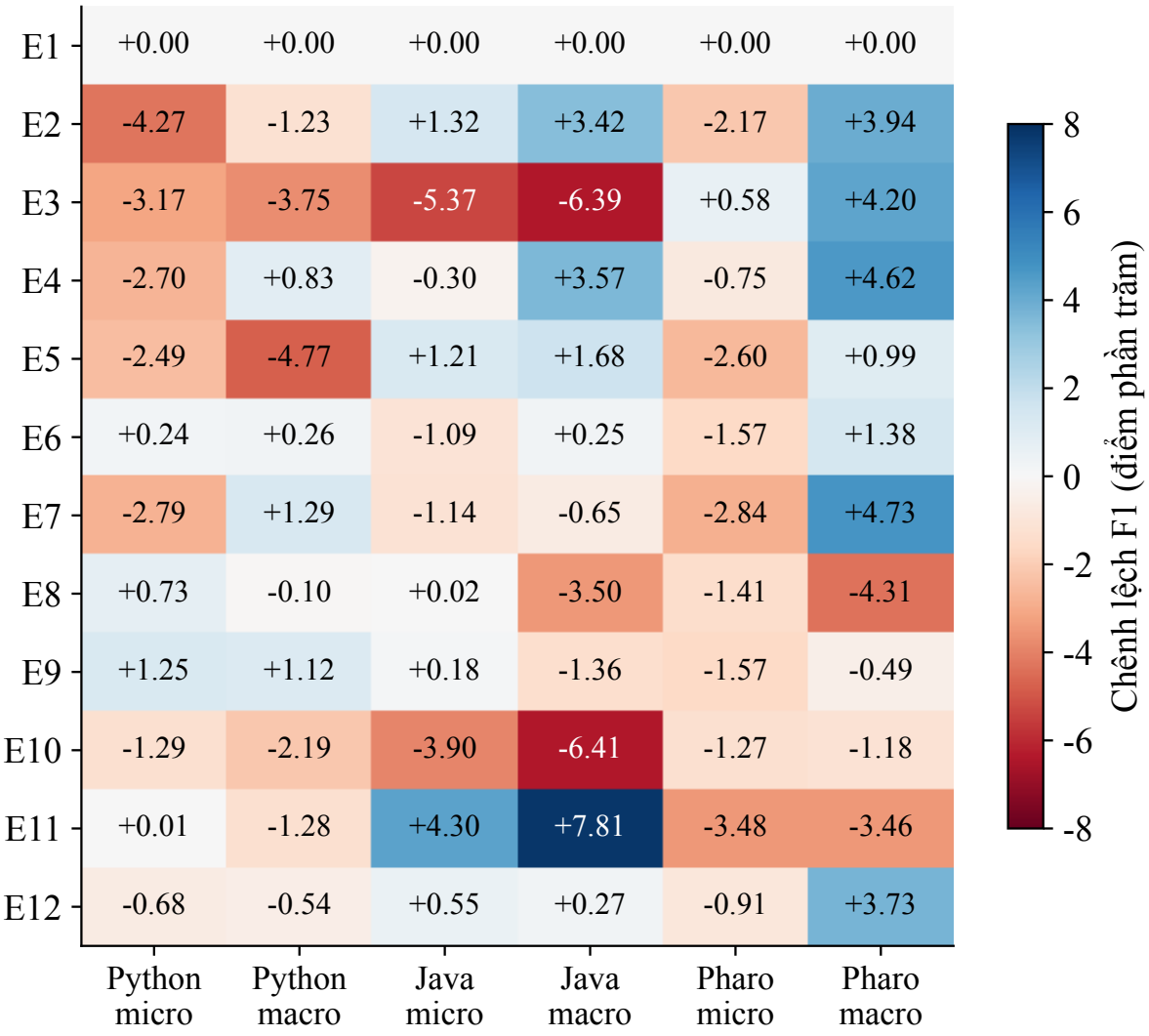
E8 đứng đầu CV 0.6029, nhưng E11 rất gần, 0.6006. Chênh 0.0024 nhỏ hơn nhiều SD fold; không có cơ sở khẳng định vượt trội thống kê. Theo tiêu chí chính, E8 vẫn là ứng viên đầu tiên; mục tiêu chi phí hoặc macro cần một quy tắc khác được khai báo rõ.

Trên test, E11 có micro F1 0.6110, macro F1 0.6778, macro AP 0.6937, đều cao nhất. So với E1, micro tăng 4.30 và macro tăng 7.81 điểm phần trăm. E8 chỉ tăng test micro khoảng 0.02 điểm nhưng giảm macro 3.50 điểm và tốn thêm thời gian. Chỉ nhìn một số CV đứng đầu sẽ che khuất đánh đổi này.

6.3 Pharo: Focal Loss cân bằng tốt hơn, ASL ưu tiên recall

E3 dẫn CV micro 0.6923 và test micro 0.7028, HL thấp nhất 0.0928. So với E1, test micro tăng 0.58 và macro tăng 4.20 điểm phần trăm. Gain micro nhỏ nên cần lặp seed, dù đi cùng cải thiện macro/HL trong run này.

E7 dẫn macro F1 0.6250 và macro AP 0.6934, nhưng micro 0.6686 thấp hơn E1. Macro của E7/E4 rất gần (0.6250/0.6239), không phải khoảng cách lớn. CodeBERT giảm cả CV và test so với E1: nhãn "chuyên code" không bảo đảm phù hợp với mọi ngôn ngữ.



Hình 3: Chênh lệch test F1 so với E1, đơn vị điểm phần trăm. Không có một phương pháp tăng đồng nhất cả micro/macro ở cả ba ngôn ngữ.

7 Phân tích cơ chế và insight

7.1 Mất cân bằng: tăng recall chưa chắc tăng hiệu quả tổng thể

Weighted BCE có lợi khác nhau: Java tăng micro/macro; Pharo tăng macro nhưng giảm micro; Python giảm cả hai trên test dù CV macro cải thiện. Có imbalance không đồng nghĩa class weight luôn tốt hơn. Trọng số thay đổi mục tiêu huấn luyện, không trực tiếp tối ưu mọi metric.

Ở Python, E2 giúp DevelopmentNotes/Expand nhưng làm giảm Parameters/Usage; hai nhãn nhiều positive tác động mạnh đến micro F1. Tại Pharo, E2 cải thiện Classreferences và Collaborators nhưng làm giảm F1 của Example và Intent.

Bảng 10: F1 test của một số nhãn minh họa; S là số positive test.

Python	S	E1	E2	E4	E7	E11
DevelopmentNotes	62	0.3182	0.3636	0.4000	0.3977	0.2299
Expand	101	0.3780	0.4061	0.3860	0.4747	0.4000
Usage	160	0.7881	0.7062	0.7881	0.7092	0.7657
Java	**S**	**E1**	**E2**	**E4**	**E7**	**E11**
Pointer	73	0.3333	0.5926	0.5714	0.4058	0.5289
rational	56	0.3721	0.2087	0.2708	0.2602	0.5977
Ownership	21	1.0000	1.0000	1.0000	1.0000	0.9756
Pharo	**S**	**E1**	**E2**	**E4**	**E7**	**E11**
Classreferences	15	0.2609	0.5143	0.4500	0.6000	0.4545
Collaborators	25	0.3500	0.4615	0.4444	0.4444	0.1818
Example	150	0.8475	0.8063	0.8293	0.7816	0.8097

ASL đẩy recall cao ở cả ba ngôn ngữ, lần lượt khoảng 0.773, 0.753 và 0.833, đi cùng precision thấp hơn E1. AP Pharo cao cho thấy thứ hạng hữu ích nhưng ngưỡng 0.5 chưa tạo micro F1 tốt nhất. Điều này gợi ý ASL cộng OOF tuning cho tương lai; không được hạ ngưỡng theo test để cải thiện bảng hiện tại. Focal Loss tốt trên Pharo nhưng giảm mạnh test Java, nên tác động loss phụ thuộc phân bố và độ khó của từng hệ nhãn, không thể suy từ tên phương pháp.

7.2 Threshold tuning: macro tăng bằng một đánh đổi đo được

E4 giữ AP giống E1 ở cả ba ngôn ngữ, nhất quán với việc chỉ đổi quyết định. Macro F1 tăng cả ba, nhưng micro đều giảm và HL đều tăng. Tối đa hóa F1 từng nhãn trên OOF không đồng nghĩa tối ưu micro/HL.

Bảng 11: Precision/recall micro và số lỗi test E1/E4. FP là false positive, FN là false negative.

Ngôn ngữ	Exp	Precision	Recall	FP	FN
Python	E1	0.7006	0.6003	147	229
Python	E4	0.5911	0.6510	258	200
Java	E1	0.6048	0.5353	198	263
Java	E4	0.4522	0.7527	516	140
Pharo	E1	0.7424	0.6569	93	140
Pharo	E4	0.6651	0.7157	147	116

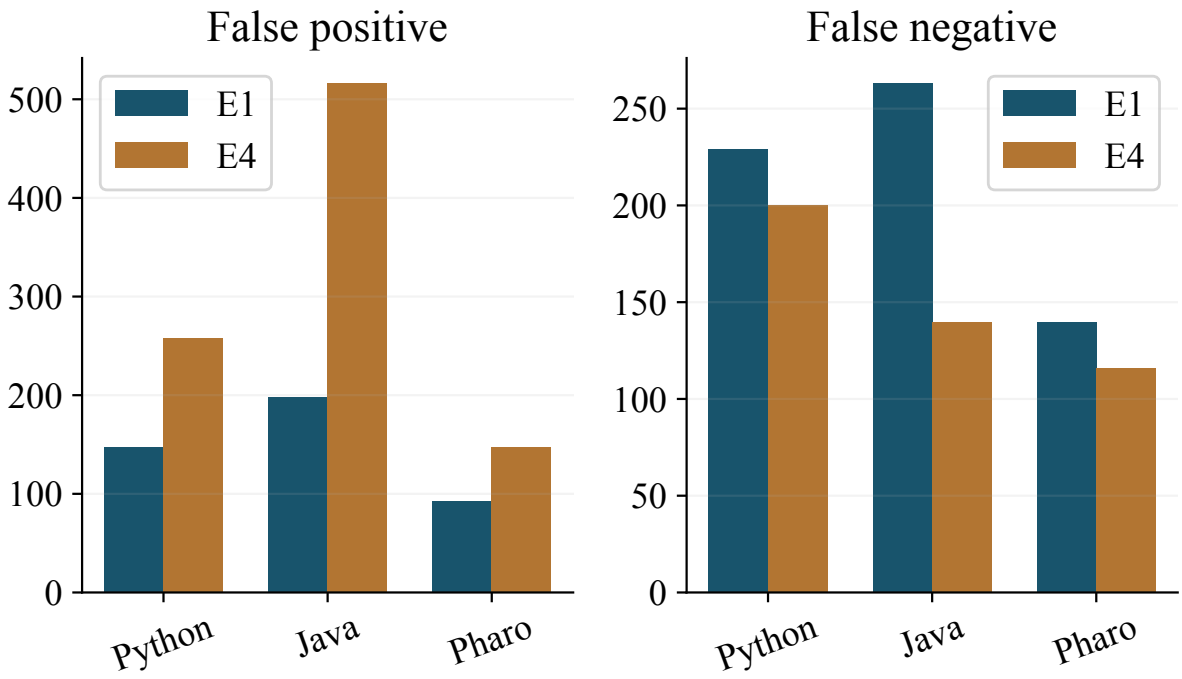
Java giảm FN từ 263 xuống 140 nhưng tăng FP từ 198 lên 516. Macro F1 tăng 0.5997 lên 0.6354, HL tăng 0.1361 lên 0.1936. Pointer được lợi (0.3333 lên 0.5714), rational giảm (khoảng 0.372 xuống 0.271). Tại Pharo, Classreferences/Collaborators tăng, Example giảm nhẹ; threshold OOF tương ứng 0.15/0.20/0.70. Ngưỡng tìm được khác nhau giữa nhãn cho thấy 0.5 không phải giả định tối ưu cho F1 riêng từng nhãn; chưa có phép đo calibration độc lập.

7.3 Attention và graph: thêm cấu trúc không bảo đảm thắng

E5 chỉ tăng test micro trên Java, giảm ở Python/Pharo. E6 cải thiện E5 trên Python/Pharo nhưng giảm trên Java. Hệ số contrastive 0.1 cố định chưa được chứng minh tối ưu; không có bằng chứng cho lợi ích phổ quát.

E12 tăng micro/macro so với E5 trên Python và Pharo, giảm cả hai trên Java. Graph chỉ có 3/3/6 cạnh và nhiều node không liên kết nhãn khác. Prior tương quan được cung cấp khá hạn chế. Một giả thuyết là graph có ích khi context và đồng xuất hiện phù hợp, nhưng có thể truyền thông tin không hữu ích hoặc làm nhãn khó tách hơn. Chưa có ablation nhiều seed để khẳng định cơ chế nhân quả.

E9 v2 có micro test tốt nhất Python, nhưng trung bình ba ngôn ngữ khoảng 0.6367, gần E1 là 0.6372. Khởi tạo head và chính sách checkpoint riêng khiến lợi ích phải được quy cho toàn bộ biến thể, không chỉ graph. Ownership Java gần hoàn hảo ở nhiều model trong khi Expand/rational khó hơn. Dấu hiệu như thẻ tác giả có thể khiến một nhãn dễ



Hình 4: E4 so với E1: FN giảm nhưng FP tăng. Số đếm được đối soát với precision/recall và HL đã lưu.

nhận diện; điểm cao đó không tự chứng minh model hiểu quan hệ nhãn phức tạp.

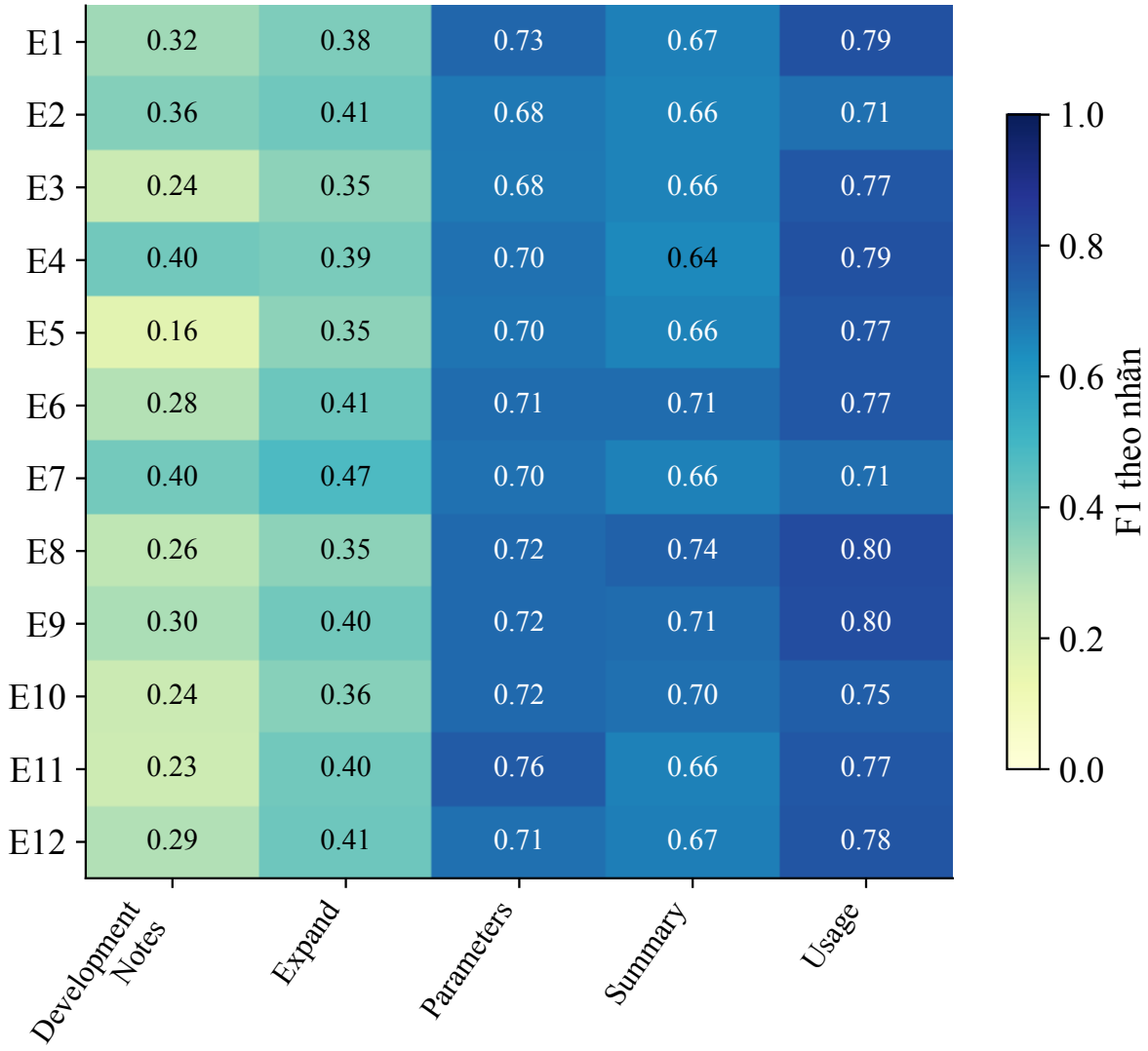
7.4 Backbone và mức độ phù hợp miền

E11 tăng CV micro so với E1 6.63 điểm trên Python, 0.97 trên Java, nhưng giảm 1.76 trên Pharo. Test Java có gain rõ, Python gần ngang, Pharo giảm 3.48 điểm micro. Phân bố ngôn ngữ pretrain là giả thuyết hợp lý, nhưng quy mô test, class distribution và siêu tham số chưa khảo sát riêng cũng có thể góp phần. SD thấp của E11 Python trong một seed không được suy rộng thành ổn định ở mọi lần train.

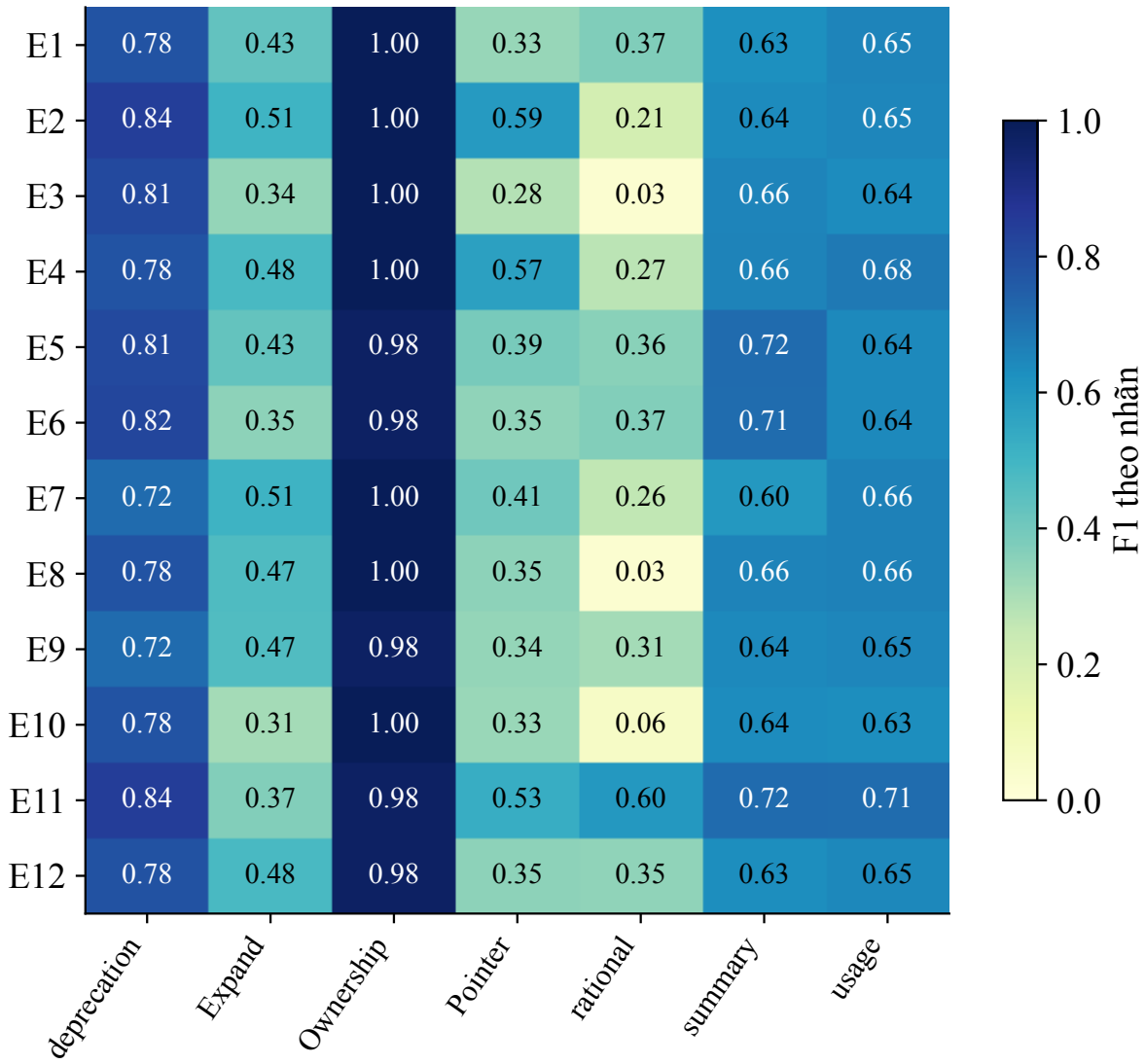
7.5 Hội tụ và lựa chọn epoch

Hình 8 vẽ từng fold riêng, không bù các epoch đã dừng và không trung bình các tập fold thay đổi theo thời gian. Một số fold đạt đỉnh sớm, một số cải thiện gần epoch 10. Không có căn cứ buộc mọi model dừng đúng 5 hoặc train cố định đủ 10 epoch.

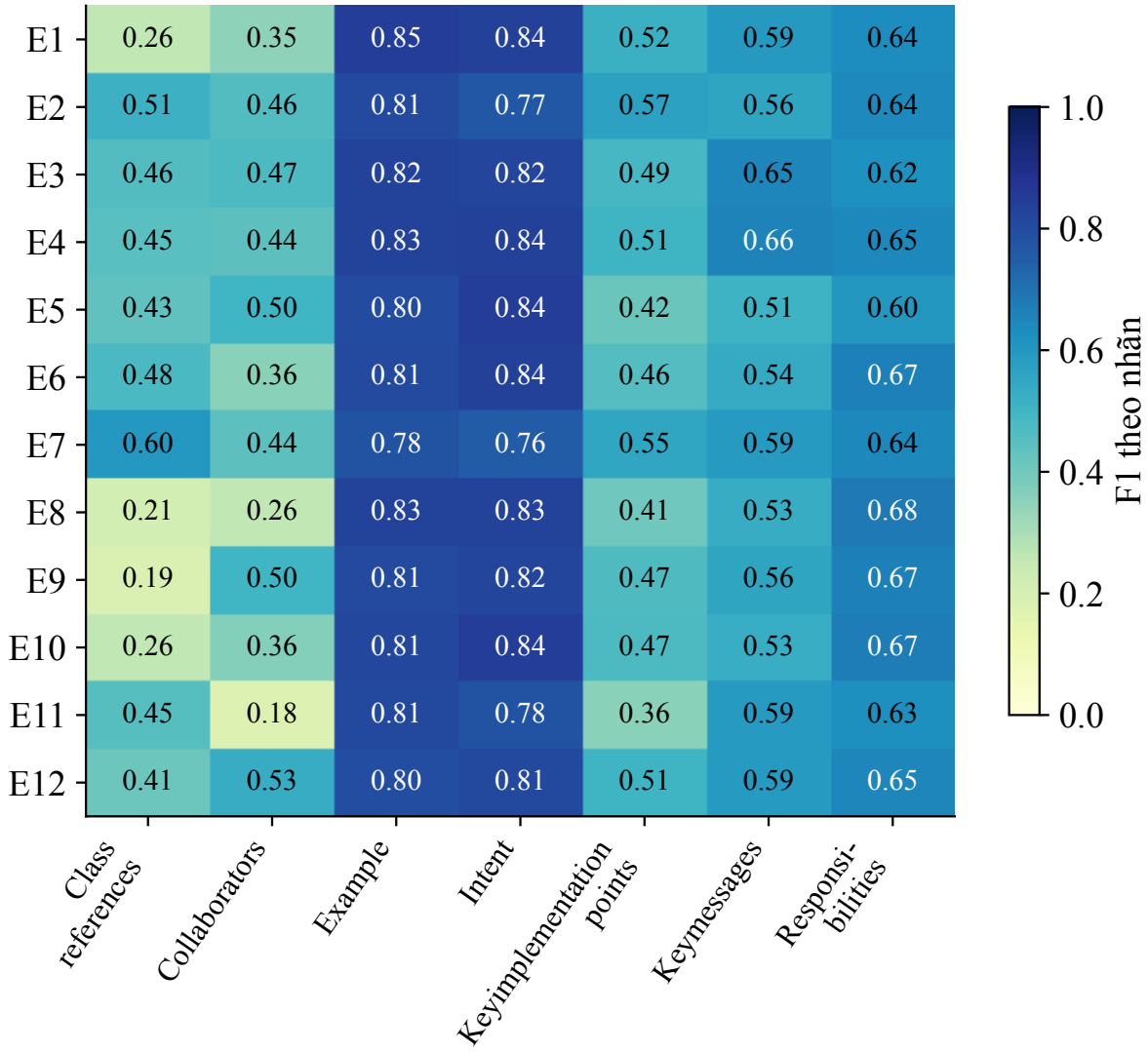
Best epoch của E11 Python là 7, 10, 6, 10, 5; E1 Python là 4, 4, 8, 8, 7. E3 Pharo có 9, 6, 10, 5, 10. Cần early stopping và khôi phục best weights thay vì dùng epoch cuối đã chạy. Giới hạn 10 chưa chứng minh hội tụ; tăng ngân sách cần kế hoạch validation mới, không dựa vào test đã xem. E9 cũ từng mắc kẹt F1 bằng 0 nên giữ epoch đầu; v2 sửa khởi tạo/thang logit và phân xử checkpoint, thay vì chỉ tăng epoch để che hiện tượng.



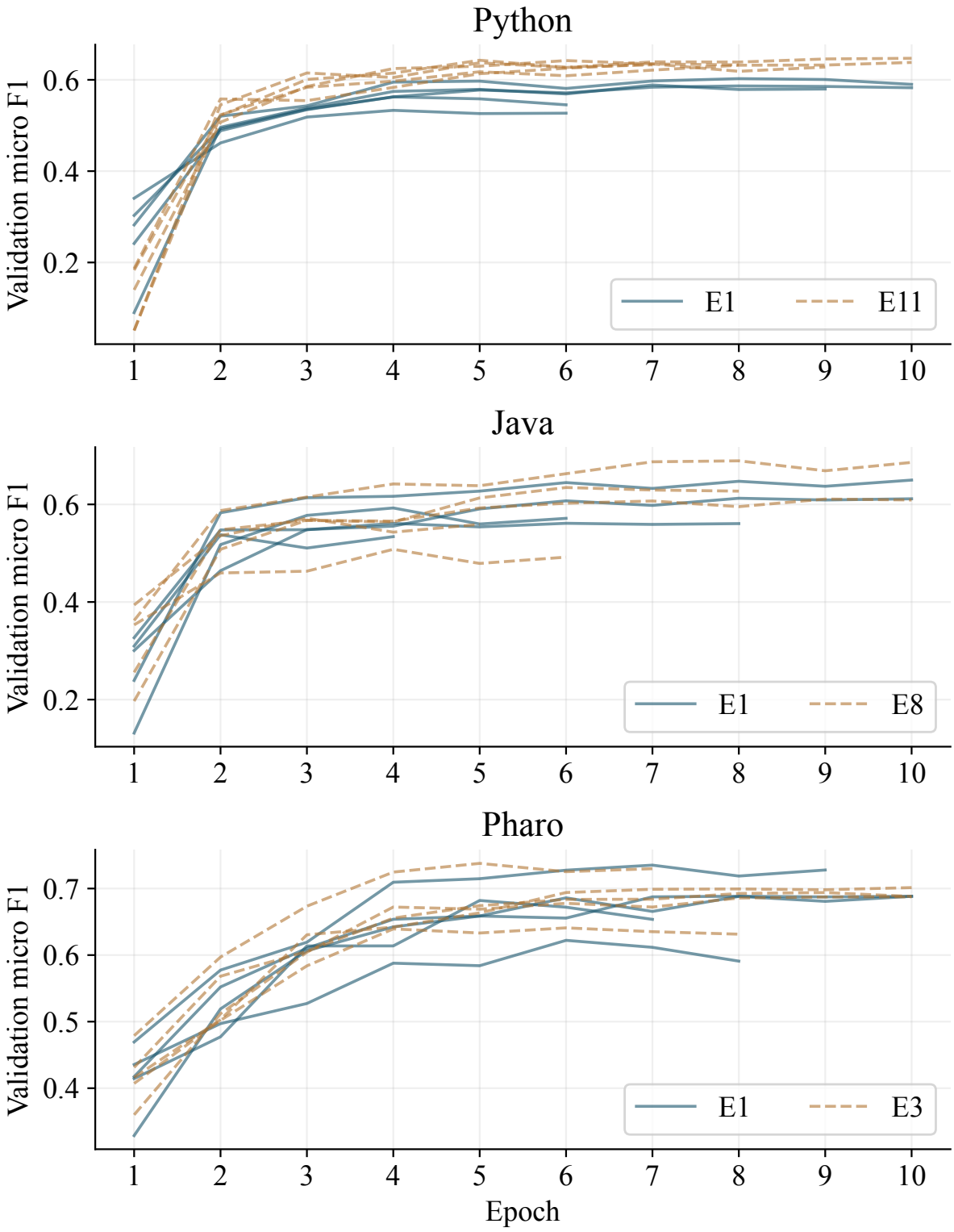
Hình 5: F1 theo nhãn Python. Các nhãn phản ứng khác nhau với từng phương pháp.



Hình 6: F1 theo nhãn Java. Ownership dễ hơn nhiều nhãn khác; CodeBERT không thắng mọi nhãn.



Hình 7: F1 theo nhãn Pharo. Macro nhạy với nhãn hiếm, micro chịu ảnh hưởng lớn của Example.



Hình 8: Validation micro F1 của E1 và ứng viên chọn bằng CV: E11 Python, E8 Java, E3 Pharo. Mỗi đường là một fold.

8 Chi phí và chiến lược lựa chọn

8.1 Chi phí pipeline quan sát được

Bảng 12: Thời gian mỗi run, phút: 5-fold CV, full train và evaluation. Không phải phép đo inference latency.

Exp	Python	Java	Pharo
E1	14.84	12.88	11.16
E2	15.69	11.13	11.60
E3	12.96	12.92	11.93
E4	15.07	13.09	11.29
E5	14.84	14.03	11.62
E6	15.47	13.74	12.01
E7	18.33	14.11	12.60
E8	27.04	23.93	17.65
E9	15.79	12.81	11.98
E10	36.98	29.33	26.37
E11	16.32	12.74	11.91
E12	17.57	12.93	12.15

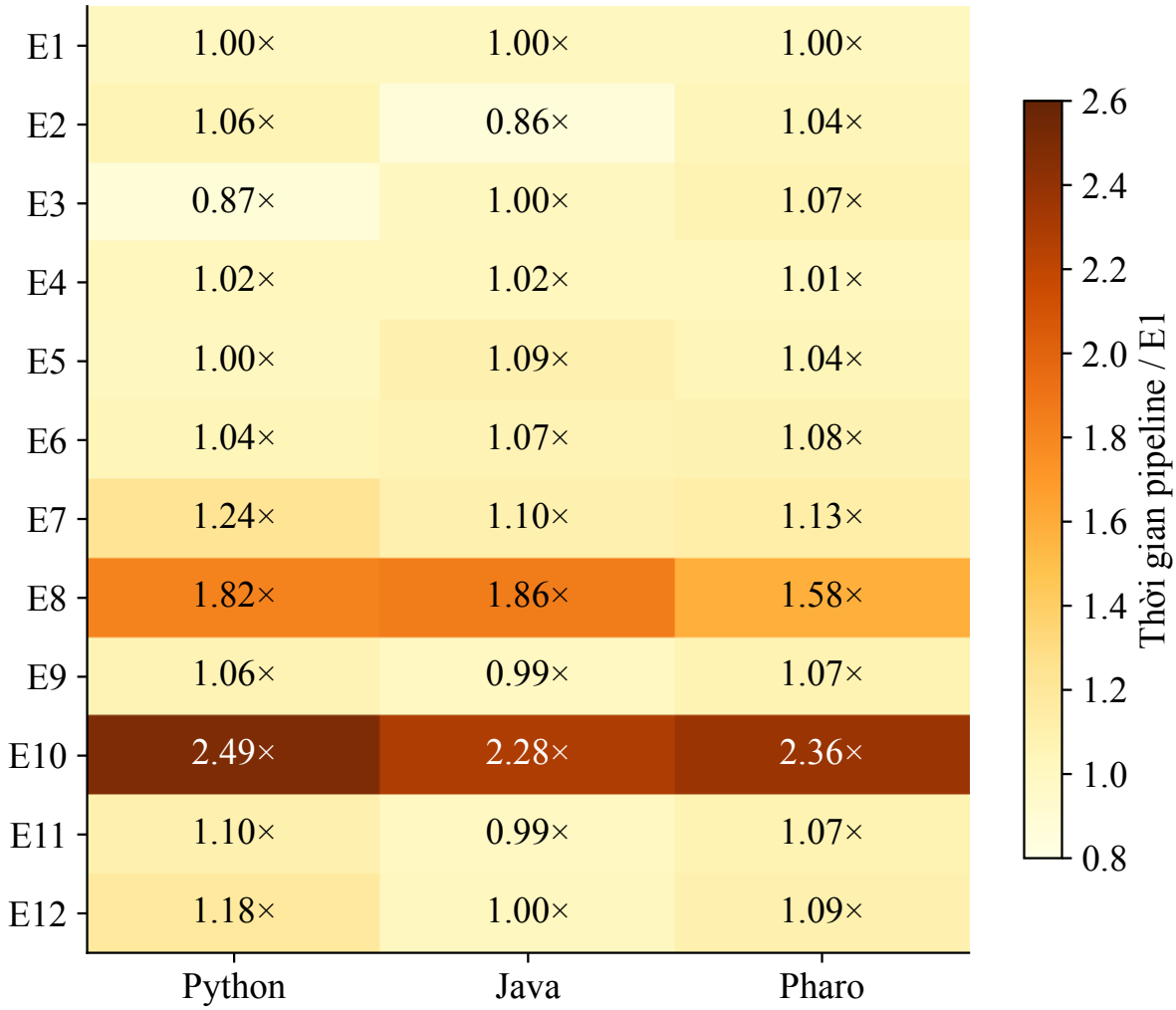
E8 cần hai forward, E10 thêm gradient embedding và forward đối kháng. Thời gian E8 quan sát bằng 1.58–1.86 lần E1, E10 bằng 2.28–2.49 lần. Tuy nhiên, epoch thực chạy, GPU, cache và chạy đồng thời khác nhau; đây là tỷ lệ pipeline, không phải benchmark tốc độ được kiểm soát.

E10 không tăng test micro/macro so với E1 ở cả ba ngôn ngữ trong cấu hình này dù tốn chi phí cao hơn. E8 cải thiện micro Python, Java gần ngang và Pharo giảm; macro không tăng. Cơ sở lý thuyết về regularization cần được kiểm chứng cùng mục tiêu và ngân sách, không tự là lý do ưu tiên một model.

8.2 Chọn bằng validation trước khi diễn giải test

Theo quy tắc đã nêu, chọn E11 Python, E8 Java và E3 Pharo. Trung bình đều test micro F1 của tổ hợp là 0.6393, E1 là 0.6372: gain chỉ khoảng 0.21 điểm phần trăm. Chọn theo CV đúng quy trình không bảo đảm thắng lớn trên một holdout. Không đổi E8 Java thành E11 chỉ vì sau đó thấy test cao hơn rồi gọi đó là lựa chọn độc lập.

Nếu cần một kỹ thuật dùng chung, E11 có trung bình CV micro cao nhất 0.6357, E1 là 0.6162. Phép tóm tắt này chấp nhận điểm yếu CodeBERT trên Pharo. Nếu ưu tiên



Hình 9: Tỷ lệ thời gian pipeline so với E1 cùng ngôn ngữ. Không được diễn giải trực tiếp thành chênh lệch inference speed.

Bảng 13: Ứng viên CV micro F1 cao nhất. Test chỉ đánh giá sau lựa chọn.

Ngôn ngữ	Model	CV micro F1 mean ± SD	Test micro F1	Test macro F1
Python	E11	0.6410 ± 0.0047	0.6467	0.5627
Java	E8	0.6029 ± 0.0680	0.5682	0.5647
Pharo	E3	0.6923 ± 0.0347	0.7028	0.6197

Bảng 14: So sánh ghép cặp với E1 trên cùng các fold. Số fold tăng là mô tả, không phải kiểm định qua nhiều seed.

Ngôn ngữ	Model	Tăng CV (điểm %)	Fold hơn E1
Python	E11	+6.63	5/5
Java	E8	+1.21	4/5
Pharo	E3	+0.91	4/5

đơn giản/chi phí, E1 vẫn hợp lý. Nếu ưu tiên nhãn hiếm, phải khai báo mục tiêu macro hoặc chi phí bỏ sót trước rồi chọn bằng validation tương ứng, không chọn theo micro rồi đổi cách diễn giải sang macro.

8.3 Ý nghĩa đối với triển khai

Chiến lược thực tế là xác định ngôn ngữ và chi phí FP/FN, chốt metric validation, rồi mới chọn mức độ phức tạp. Khi bỏ sót nhãn hiếm có giá cao, E2/E7/E4 là các hướng cần khảo sát; khi FP gây phiền cho người dùng, precision/HL phải được đọc cùng F1. Chỉ giữ graph/attention khi lợi ích ổn định qua nhiều seed/split bù được chi phí bảo trì.

Cần tách bốn câu hỏi: thứ hạng có tốt không (AP), quyết định ở ngưỡng cố định có tốt không (F1), nhãn nào được lợi (per-label/macro), và đổi lại bao nhiêu chi phí. Một bảng xếp hạng chỉ có một cột không trả lời được các câu hỏi này.

9 Giới hạn, độ tin cậy và khả năng tái lập

1. **Protocol khác nguồn.** Split theo group phục vụ so sánh multi-label/k-fold; chưa có phép so sánh trực tiếp với Random Forest chính thức dưới cùng test.
2. **Một seed và nhiều so sánh.** SD fold không thay thế lặp seed; chưa đủ kiểm định để khẳng định các gain nhỏ có ý nghĩa thống kê.
3. **Giới hạn tổng quát.** Class giao nhau, near-duplicate chưa loại hết, group lớn và text giống nhưng nhãn khác có thể ảnh hưởng kết quả.
4. **Selection và tuning.** CV chọn epoch trên validation; E4 dùng OOF chọn threshold nên OOF sau tuning không độc lập. E9 có chính sách checkpoint riêng.
5. **Tính thăm dò.** Test đã được xem trong quá trình phân tích; sự cố E9 cũng được nhận ra cùng validation toàn âm rồi sửa thiết kế. Không mô tả test hiện tại là benchmark mù chưa từng quan sát. Cần holdout mới hoặc nested/grouped evaluation để xác nhận kết luận.

6. **Cấu hình chưa tối ưu toàn diện.** Hệ số loss, KL, contrastive, threshold graph và FGM chủ yếu cố định; chưa có tìm kiếm hệ thống.
7. Đầu vào/chi phí. Truncation được kiểm toán ở Mục 2.3; chỉ dùng câu. Chưa đo mất mát ngữ nghĩa hoặc latency/bộ nhớ có kiểm soát.

Repository giao nộp có registry, loader processed, vòng training và metric chung. Mỗi run giữ cấu hình thực tế, nhãn, folds, seed, fingerprint, checkpoint và metric để truy vết. Dependencies cho training/tiền xử lý đã được khai báo; kiểm tra tái lập không ghi đè dữ liệu. Báo cáo không chèn source code; hướng dẫn chạy nằm trong repository kèm theo.

10 Kết luận và đối chiếu đề bài

Đề tài đã triển khai bài toán multi-label 6.738 câu, so sánh 12 cấu hình DL với baseline RoBERTa, dùng cùng bộ chia 5 fold, chạy đủ 36 run và phân tích tổng thể, theo nhãn, hội tụ, chi phí. Theo CV, lựa chọn là E11/E8/E3 cho Python/Java/Pharo; held-out test cho thấy các đánh đổi và độ bất định quan trọng hơn xếp hạng đơn lẻ.

Thay loss/threshold thường dịch precision–recall; thêm cấu trúc nhãn không bảo đảm cải thiện; pretraining cần phù hợp miền; và regularization cần được đánh giá cùng ngân sách. E1 vẫn cạnh tranh. Các cấu hình mới cần chọn theo mục tiêu ứng dụng và xác nhận qua nhiều seed/split mới.

Bảng 15: Đối chiếu yêu cầu Project 2 và nội dung thực hiện.

Yêu cầu	Minh chứng
NLBSE'23, 6.738 mẫu	Nguồn, thống kê và kiểm tra ở Mục 1–3.
Multi-label	Sigmoid, vector target và hệ nhãn 5/7/7.
ML hoặc DL classifiers	12 cấu hình Deep Learning ở Mục 4.
Metric phù hợp	Micro/macro F1, precision/recall, AP, HL.
K-fold CV	5 fold cố định, ràng buộc group chung.
Chọn kỹ thuật phù hợp	Quy tắc CV và mục tiêu ứng dụng ở Mục 8.
Chạy đánh giá	36 run hoàn chỉnh, bảng ba ngôn ngữ.
Baseline RoBERTa	E1 đối chứng cùng split và đầu vào.
Phân tích kết quả	Cơ chế, nhãn khó, threshold, graph, hội tụ, chi phí, giới hạn.

Tài liệu tham khảo

[1] R. Kallis và cộng sự. *The NLBSE'23 Tool Competition*. NLBSE 2023. https://nlbse2023.github.io/tools/.

[2] NLBSE'23. *Code Comment Classification: dataset and baselines*. https://github.com/nlbse2023/code-comment-classification.

[3] Y. Liu và cộng sự. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. 2019. https://arxiv.org/abs/1907.11692.

[4] T.-Y. Lin và cộng sự. *Focal Loss for Dense Object Detection*. ICCV 2017. https://arxiv.org/abs/1708.02002.

[5] E. Ben-Baruch và cộng sự. *Asymmetric Loss For Multi-Label Classification*. ICCV 2021. https://arxiv.org/abs/2009.14119.

[6] X. Liang và cộng sự. *R-Drop: Regularized Dropout for Neural Networks*. NeurIPS 2021. https://arxiv.org/abs/2106.14448.

[7] Z.-M. Chen và cộng sự. *Multi-Label Image Recognition with Graph Convolutional Networks*. CVPR 2019. https://arxiv.org/abs/1904.03582.

[8] T. Miyato, A. M. Dai và I. Goodfellow. *Adversarial Training Methods for Semi-Supervised Text Classification*. ICLR 2017. https://arxiv.org/abs/1605.07725.

[9] Z. Feng và cộng sự. *CodeBERT: A Pre-Trained Model for Programming and Natural Languages*. Findings of EMNLP 2020. https://arxiv.org/abs/2002.08155.

[10] Microsoft. *CodeBERT: model and pretraining data documentation*. https://github.com/microsoft/CodeBERT.